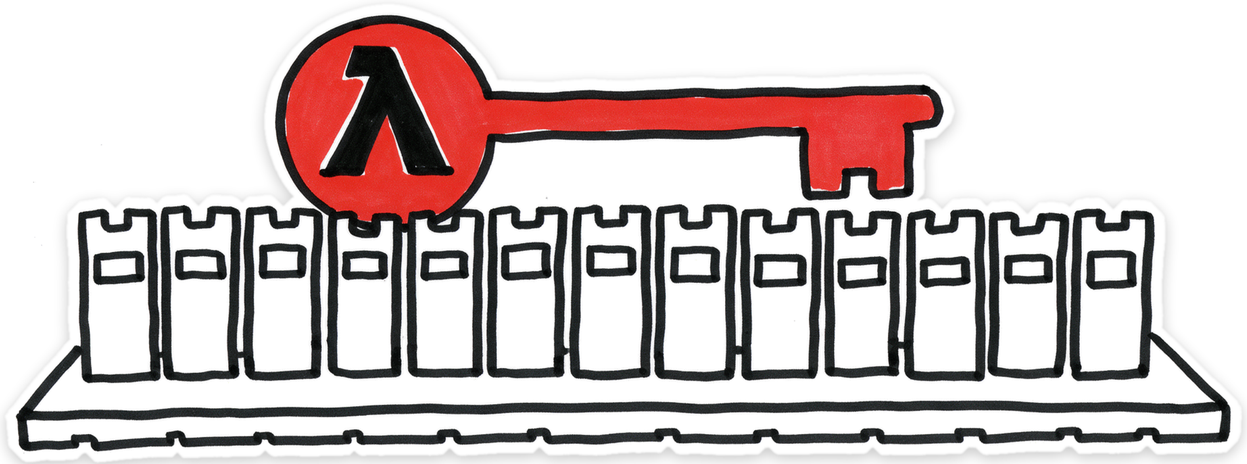
# Ключ от библиотеки

Словари и множества



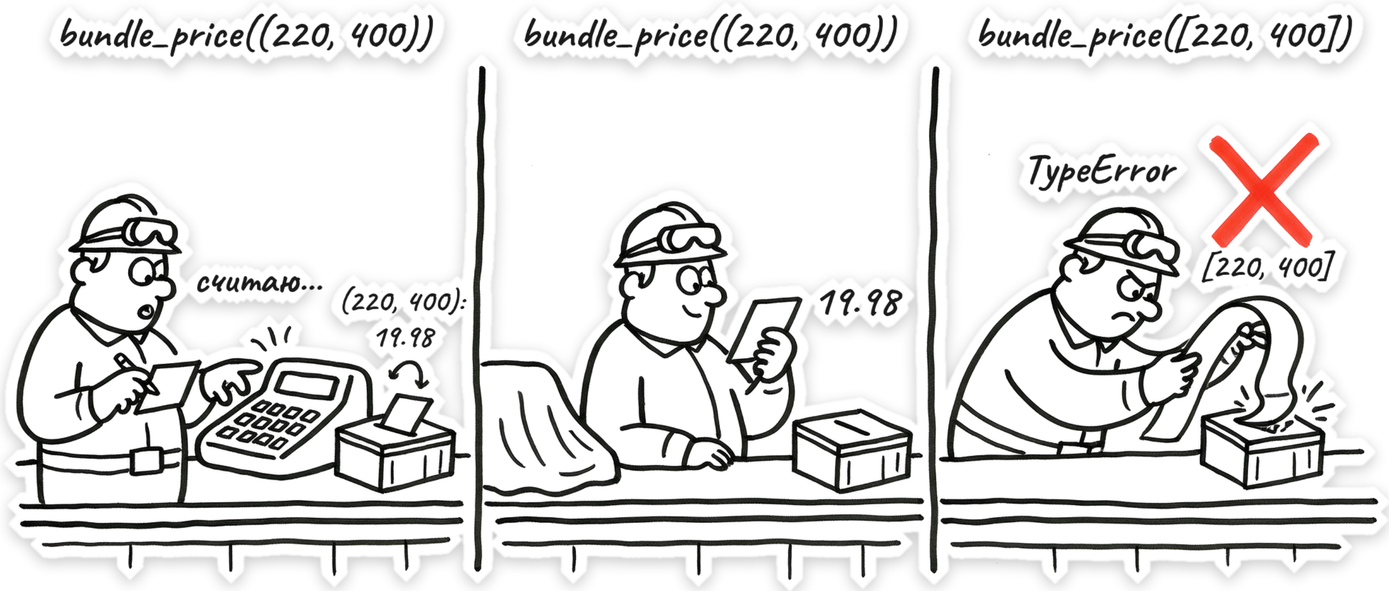
# Сколько раз посчитаем цену?

```python
from functools import cache

@cache
def bundle_price(app_ids):
    print("считаю...")
    return 9.99 * len(app_ids)

print(bundle_price((220, 400)))
print(bundle_price((220, 400)))
print(bundle_price([220, 400]))
```

# Кэш хранит результаты в словаре

```
from functools import cache

@cache
def bundle_price(app_ids):
    print("считаю...")
    return 9.99 * len(app_ids)

print(bundle_price((220, 400)))
print(bundle_price((220, 400)))
print(bundle_price([220, 400]))
```

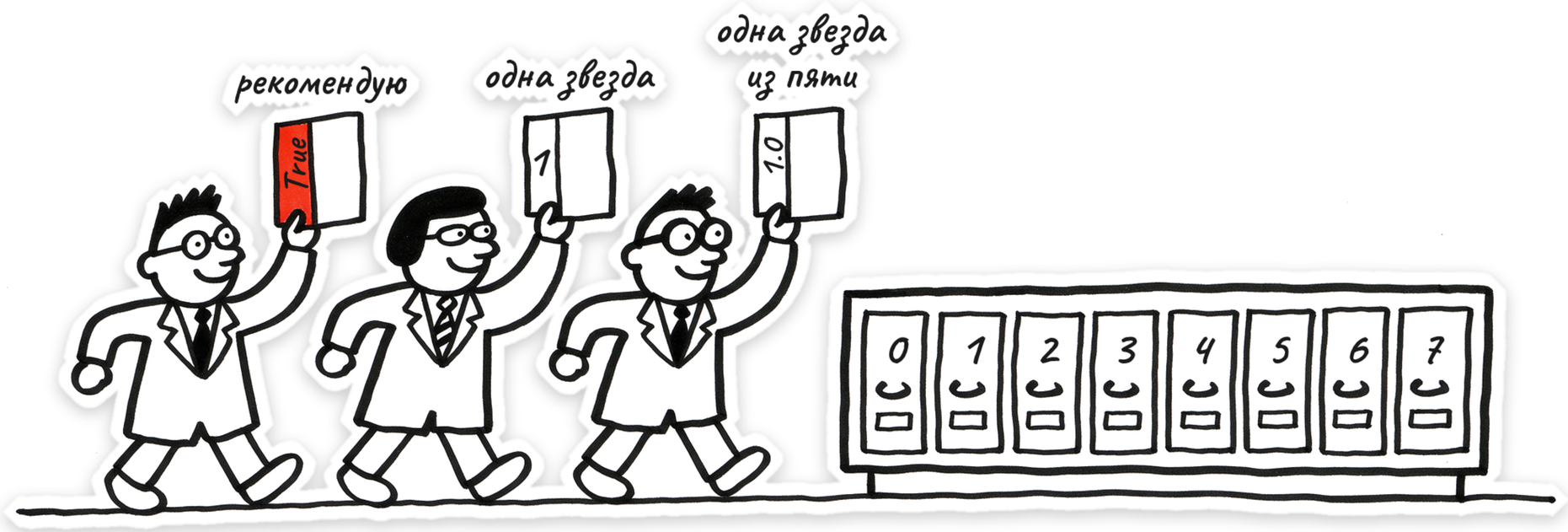
```
считаю...
19.98
19.98
TypeError: unhashable type: 'list'
```



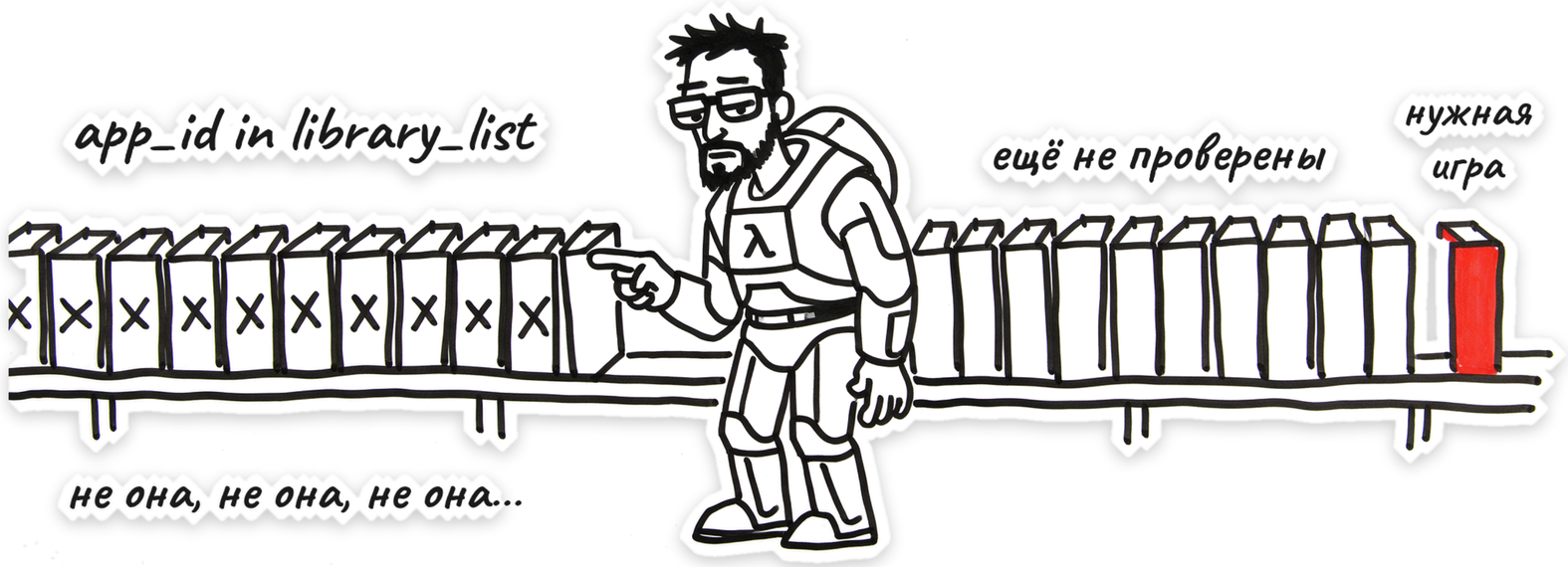

Ключ кэша — аргументы вызова

# Сколько отзывов осталось?

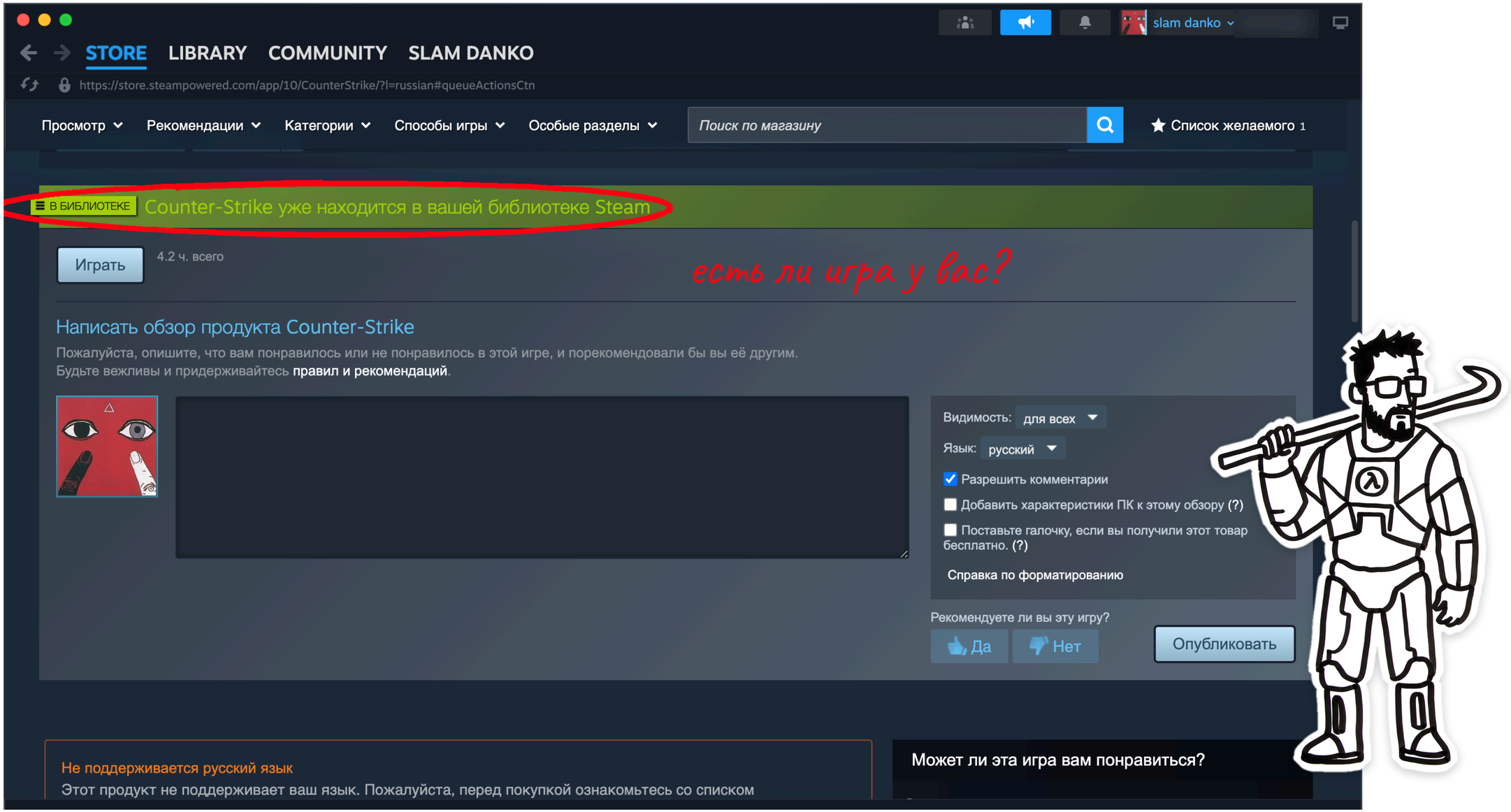
```
review = {True: "рекомендую", 1: "одна звезда", 1.0: "одна звезда из пяти"}
print(review)
print(len({1, 1.0, True}))
```



# 1
# Цена поиска



# Игра уже у вас?

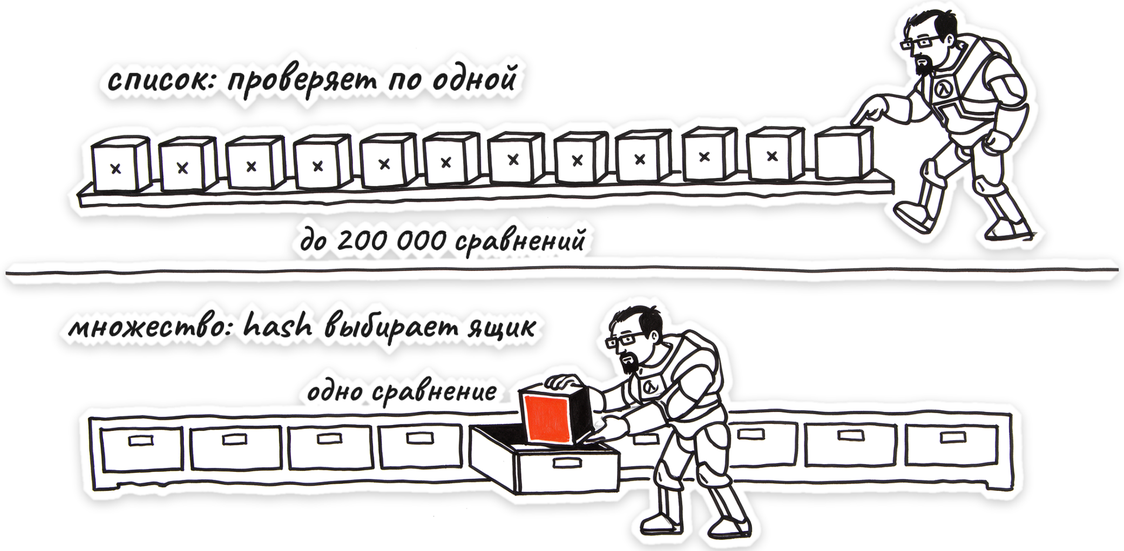


# Список или множество?

```
import random
import time

random.seed(8)
library_list = list(range(200_000))
library_set = set(library_list)
visits = [random.randrange(400_000) for _ in range(1000)]

start = time.perf_counter()
badges = sum(app_id in library_list for app_id in visits)
list_time = time.perf_counter() - start

start = time.perf_counter()
badges_again = sum(app_id in library_set for app_id in visits)
set_time = time.perf_counter() - start

print(badges == badges_again)
print(f"список: {list_time:.3f} с")
print(f"множество: {set_time * 1000:.3f} мс")
```

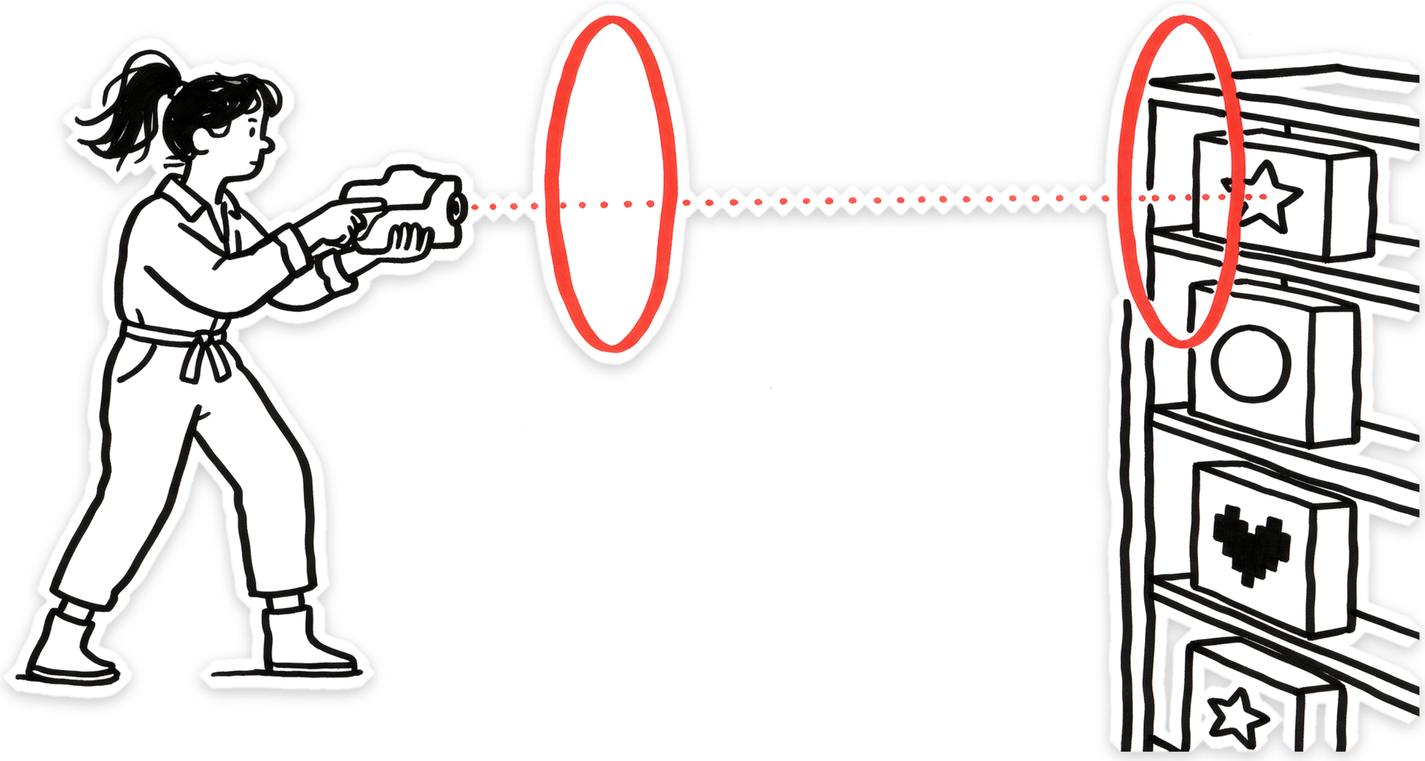
# Одна целая секунда против долей миллисекунды

```python
import random
import time

random.seed(8)
library_list = list(range(200_000))
library_set = set(library_list)
visits = [random.randrange(400_000) for _ in range(1000)]

start = time.perf_counter()
badges = sum(app_id in library_list for app_id in visits)
list_time = time.perf_counter() - start

start = time.perf_counter()
badges_again = sum(app_id in library_set for app_id in visits)
set_time = time.perf_counter() - start

print(badges == badges_again)
print(f"список: {list_time:.3f} с")
print(f"множество: {set_time * 1000:.3f} мс")
```




```
True
список: 0.940 с
множество: 0.420 мс
```

# 2
# Хеш-таблица

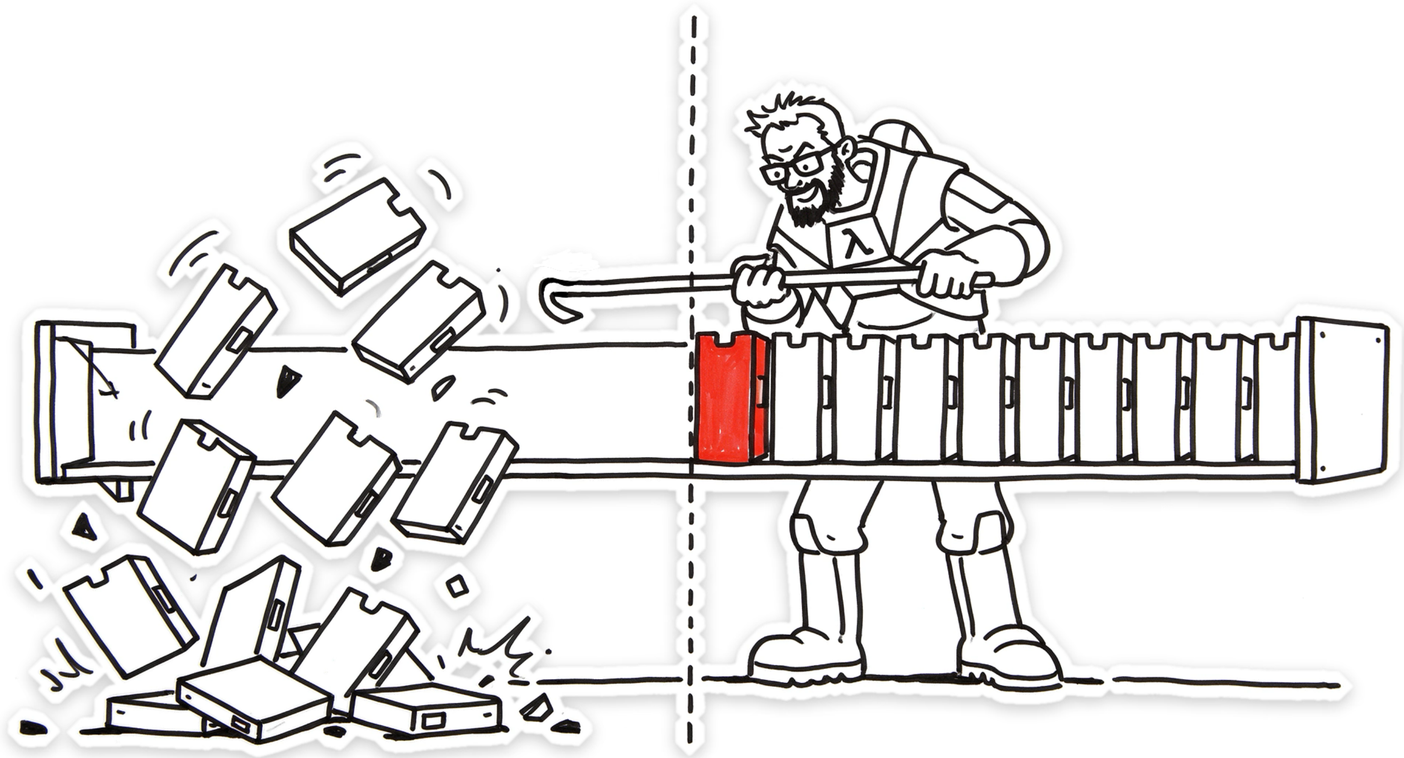


# Сколько шагов нужно пройти?

```
library = list(range(0, 400_000, 2))
target = 273_958

def binary_search(items, target):
    low, high = 0, len(items) - 1
    steps = 0
    while low <= high:
        steps += 1
        middle = (low + high) // 2
        if items[middle] == target:
            return middle, steps
        if items[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return None, steps

print(len(library))
print(binary_search(library, target))
print(binary_search(library, 273_959))
```

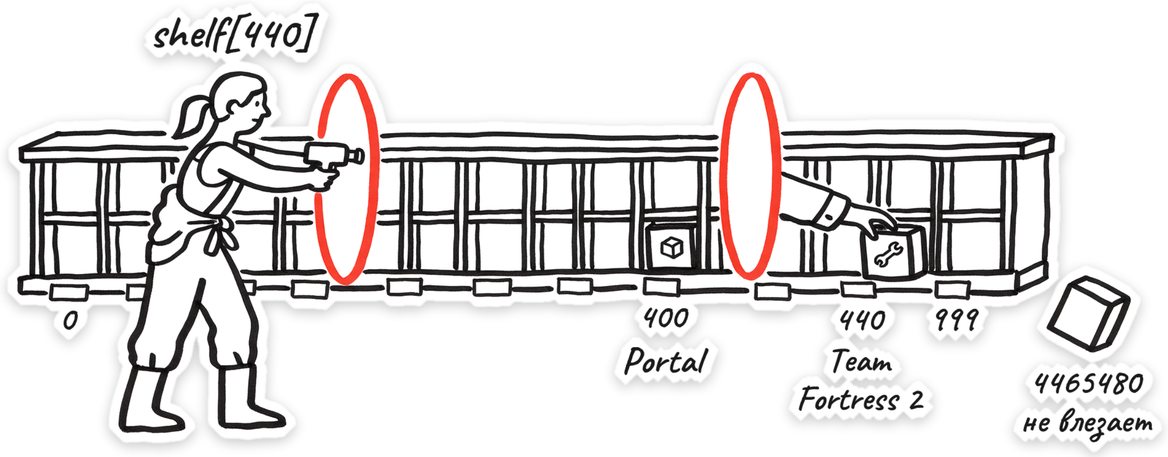
# 15 шагов вместо 136 980

```
library = list(range(0, 400_000, 2))
target = 273_958

def binary_search(items, target):
    low, high = 0, len(items) - 1
    steps = 0
    while low <= high:
        steps += 1
        middle = (low + high) // 2
        if items[middle] == target:
            return middle, steps
        if items[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return None, steps

print(len(library))
print(binary_search(library, target))
print(binary_search(library, 273_959))
```



```
200000
(136979, 15)
(None, 17)
```

# Прямой адрес



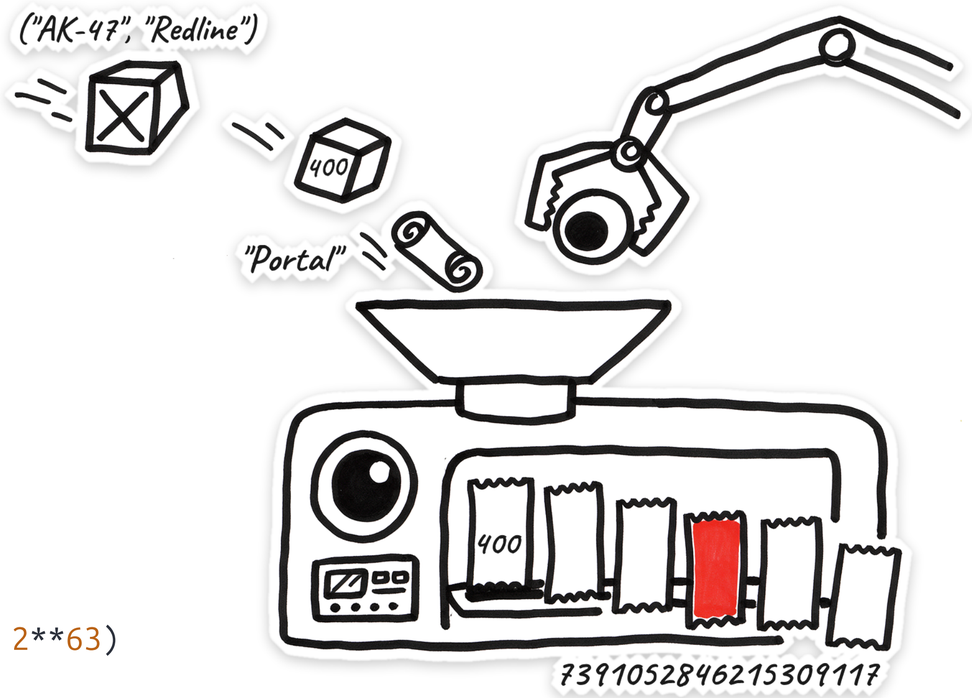

```python
shelf = [None] * 1000
shelf[400] = "Portal"
shelf[440] = "Team Fortress 2"
print(shelf[440])

try:
    shelf[4465480] = "Counter-Strike:Global Offensive"
except IndexError as error:
    print("IndexError:", error)
try:
    shelf["Portal"]
except TypeError as error:
    print("TypeError:", error)
```

```
Team Fortress 2
IndexError: list assignment index out of range
TypeError: list indices must be integers or slices, not str
```

# hash превращает объект в число

```
400
-8855365057301074806
True
True
False
<class 'int'> True
```

```
print(hash(400))
print(hash("Portal"))
print(hash("Portal") == hash("Portal"))
print(hash(("AK-47", "Redline")) == hash(("AK-47", "Redline")))
print(hash("Portal") == hash("Patrol"))
print(type(hash("Portal" * 1000)), abs(hash("Portal" * 1000)) < 2**63)
```



# Шкаф на восемь ящиков: куда лягут игры?

```
drawers = [None] * 8
for app_id in [220, 400, 570]:
    number = hash(app_id) % 8
    print(app_id, "→ hash", hash(app_id), "→ ящик", number)
    drawers[number] = app_id
print(drawers)
```

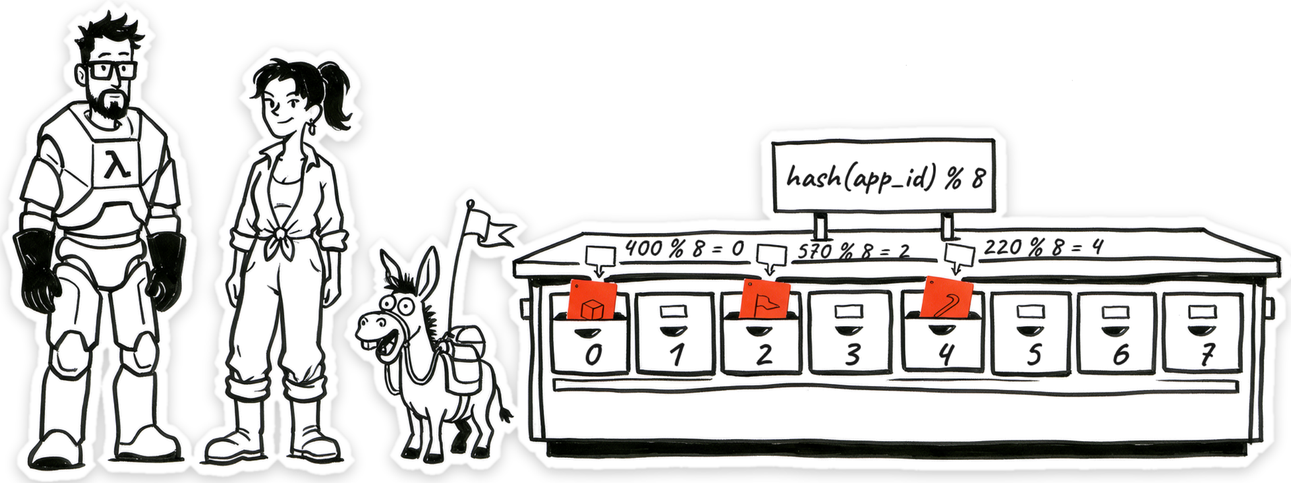
номер ящика = hash(app_id) % 8

# Каждая игра — в свой ящик

```
drawers = [None] * 8
for app_id in [220, 400, 570]:
    number = hash(app_id) % 8
    print(app_id, "→ hash", hash(app_id), "→ ящик", number)
    drawers[number] = app_id
print(drawers)
```

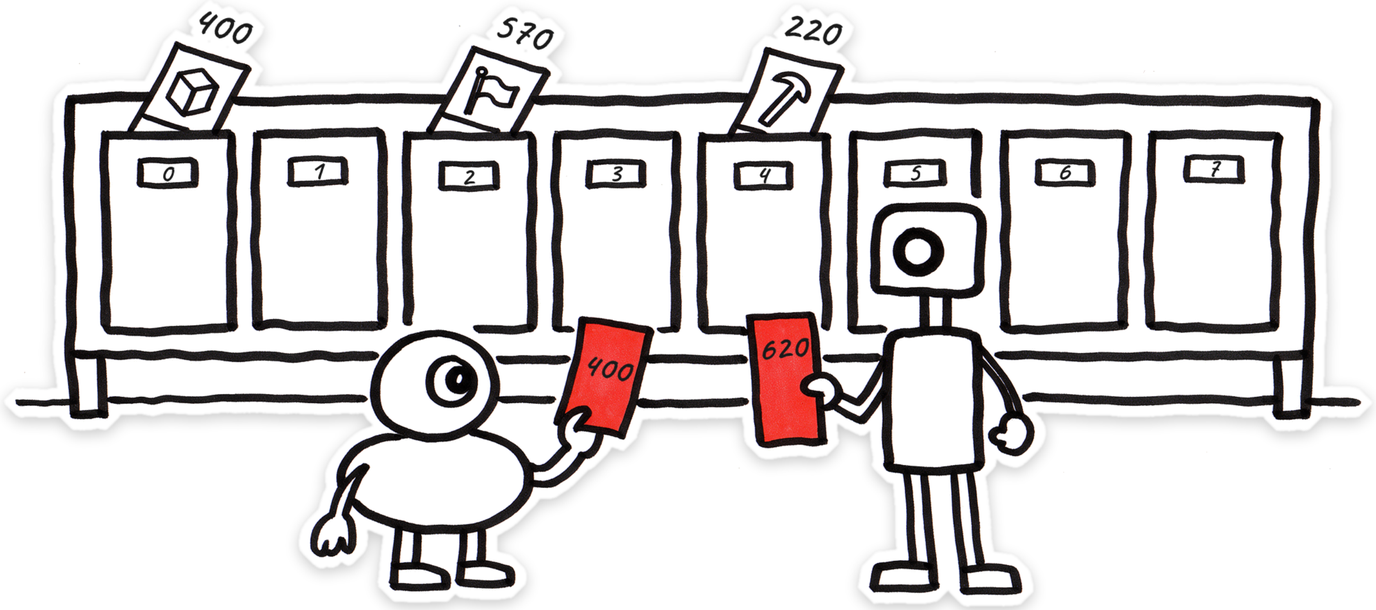
```
220 → hash 220 → ящик 4
400 → hash 400 → ящик 0
570 → hash 570 → ящик 2
[400, None, 570, None, 220, None, None, None]
```

номер ящика = hash(app_id) % 8



# Есть ли Portal? А Portal 2?

```
drawers = [400, None, 570, None, 220, None, None, None]
for app_id in [400, 620]:
    number = hash(app_id) % 8
    print(app_id, "→ ящик", number, "→ там", drawers[number],
          "→", drawers[number] == app_id)
```

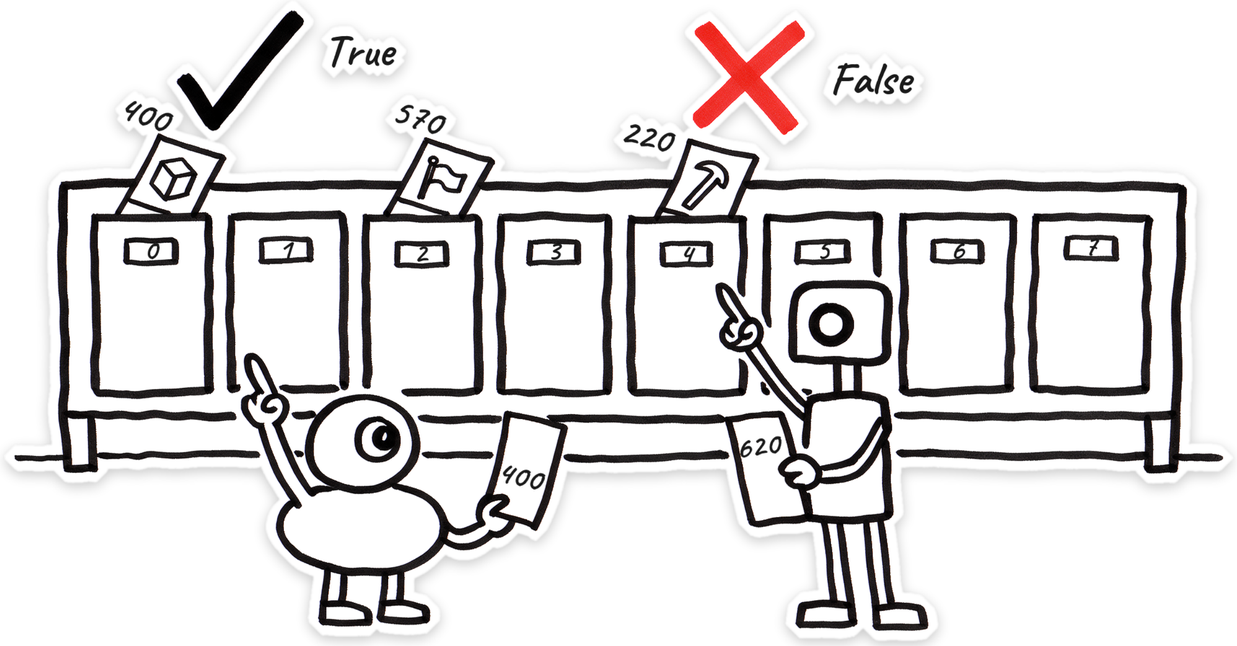


# Один ящик, одно сравнение

```
drawers = [400, None, 570, None, 220, None, None, None]
for app_id in [400, 620]:
    number = hash(app_id) % 8
    print(app_id, "→ ящик", number, "→ там", drawers[number],
          "→", drawers[number] == app_id)
```

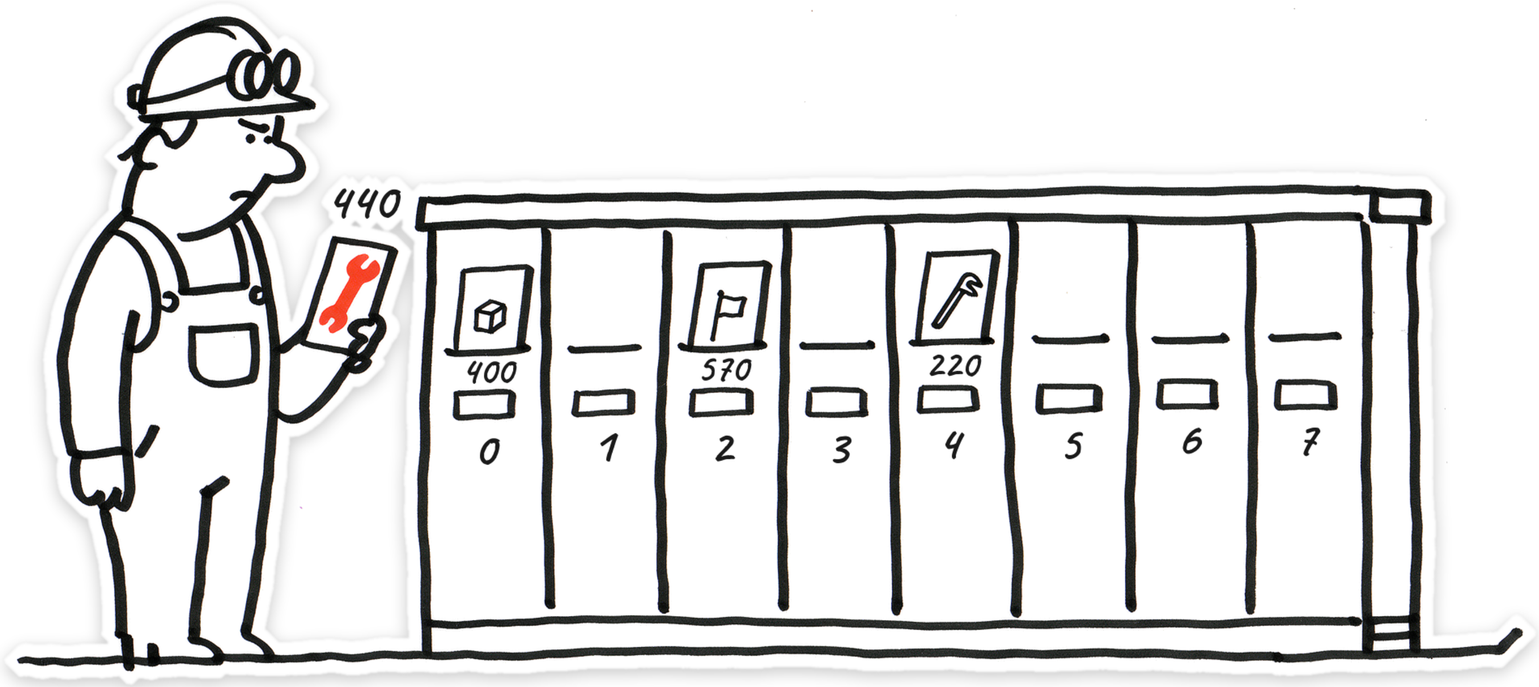
```
400 → ящик 0 → там 400 → True
620 → ящик 4 → там 220 → False
```

1 hash выбирает ящик
2 == проверяет, та ли игра



# Пришёл Team Fortress 2. Portal на месте?

```
drawers = [400, None, 570, None, 220, None, None, None]
drawers[hash(440) % 8] = 440            # Team Fortress 2
print(drawers)
print(drawers[hash(400) % 8] == 400)    # Portal ещё в библиотеке?
```



# Коллизия: разные ключи, один ящик

```
drawers = [400, None, 570, None, 220, None, None, None]
drawers[hash(440) % 8] = 440            # Team Fortress 2
print(drawers)
print(drawers[hash(400) % 8] == 400)    # Portal ещё в библиотеке?
```

```
[440, None, 570, None, 220, None, None, None]
False
```

400 % 8 = 0
440 % 8 = 0

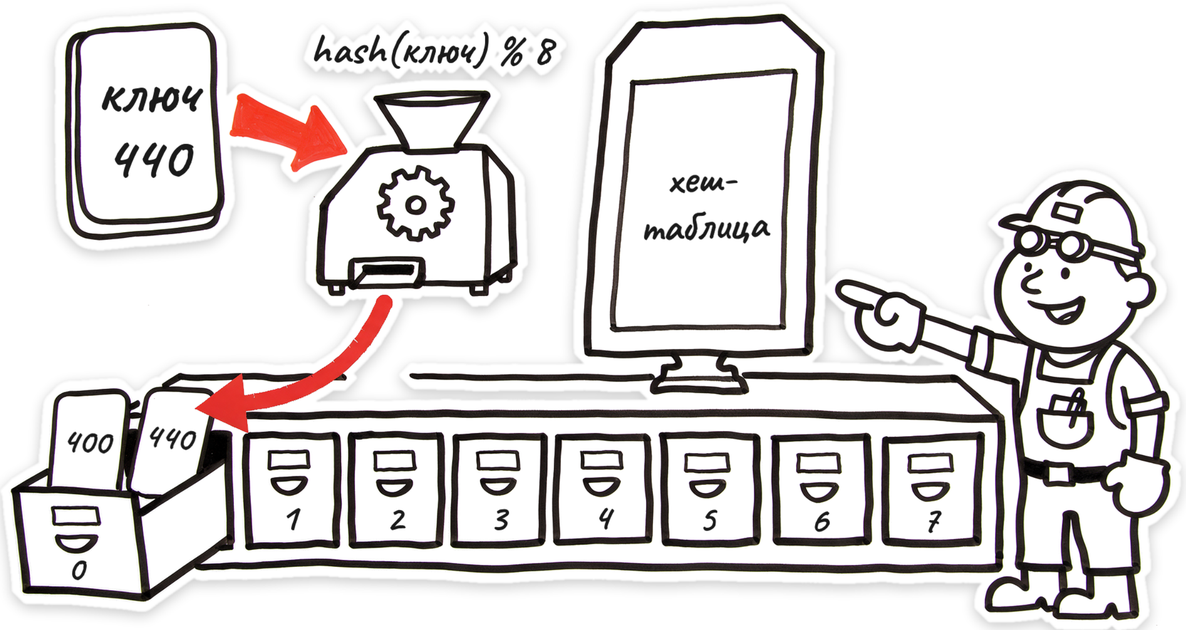
# В ящике — список

```
drawers = [[] for _ in range(8)]
for app_id in [220, 400, 570, 440, 500, 730]:
    drawers[hash(app_id) % 8].append(app_id)
print(drawers)

wanted = 440
for app_id in drawers[hash(wanted) % 8]:
    print(app_id, "==", wanted, app_id == wanted)
```

```
[[400, 440], [], [570, 730], [], [220, 500], [], [], []]
400 == 440 False
440 == 440 True
```

# Хеш-таблица

> список ящиков
> номер ящика = hash(ключ) % число ящиков
> положить: в свой ящик
> найти: свой ящик, потом ==
> set: в ящиках **ключи**
> dict: в ящиках пары **ключ — значение**



# Сколько пар в ящике 2?

```
def put(table, key, value):
    drawer = table[hash(key) % len(table)]
    for pair in drawer:
        if pair[0] == key:
            pair[1] = value
            return
    drawer.append([key, value])

table = [[] for _ in range(8)]
put(table, 400, "Portal")
put(table, 440, "Team Fortress 2")
put(table, 730, "Counter-Strike: Global Offensive")
put(table, 730, "Counter-Strike 2")
print(table[0], table[2])
```

# Ключ тот же — меняется значение

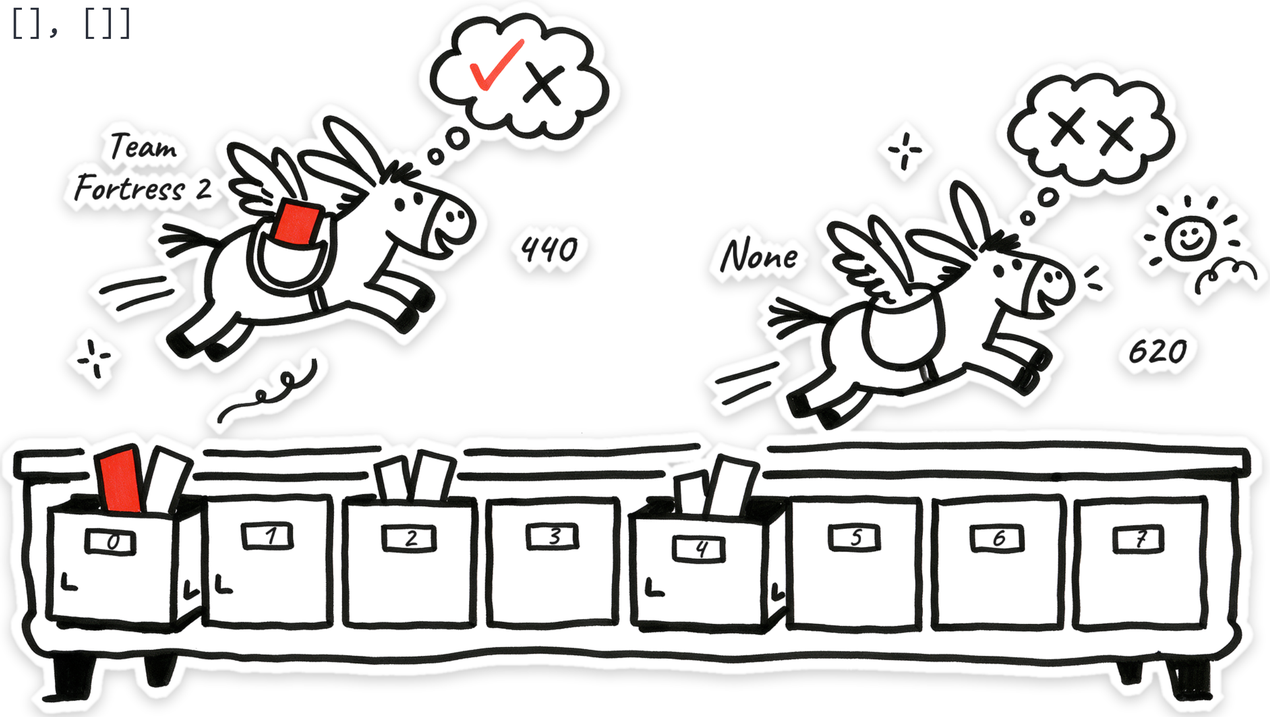
```
def put(table, key, value):
    drawer = table[hash(key) % len(table)]
    for pair in drawer:
        if pair[0] == key:
            pair[1] = value
            return
    drawer.append([key, value])

table = [[] for _ in range(8)]
put(table, 400, "Portal")
put(table, 440, "Team Fortress 2")
put(table, 730, "Counter-Strike: Global Offensive")
put(table, 730, "Counter-Strike 2")
print(table[0], table[2])
```

```
[[400, 'Portal'], [440, 'Team Fortress 2']] [[730, 'Counter-Strike 2']]
```

# get: тот же ящик, потом ==

```
def get(table, key):
    for pair in table[hash(key) % len(table)]:
        print("  сравниваю", pair[0], "==", key)
        if pair[0] == key:
            return pair[1]
    return None

table = [[[400, "Portal"], [440, "Team Fortress 2"]], [],
         [[570, "Dota 2"], [730, "Counter-Strike 2"]], [],
         [[220, "Half-Life 2"], [500, "Left 4 Dead"]], [], [], []]
print(get(table, 440))
print(get(table, 620))
```

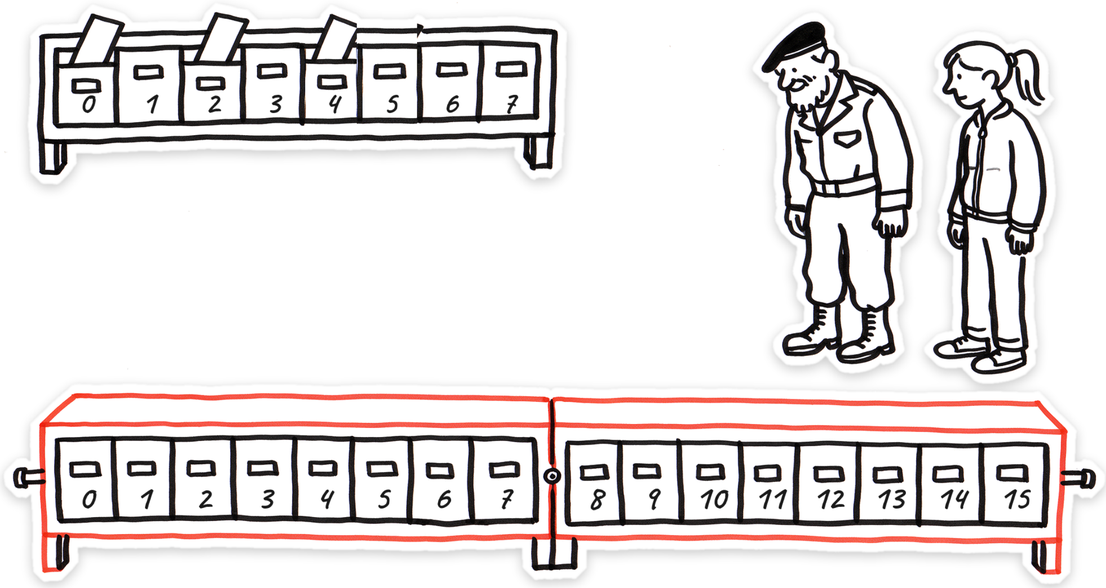
```
  сравниваю 400 == 440
  сравниваю 440 == 440
Team Fortress 2
  сравниваю 220 == 620
  сравниваю 500 == 620
None
```



# Шкаф на 16 ящиков. Кто остался соседями?

```
old = [[400, 440], [], [570, 730], [], [220, 500], [], [], []]
new = [[] for _ in range(16)]
for drawer in old:
    for app_id in drawer:
        new[hash(app_id) % 16].append(app_id)

for number, drawer in enumerate(new):
    if drawer:
        print(number, drawer)
```

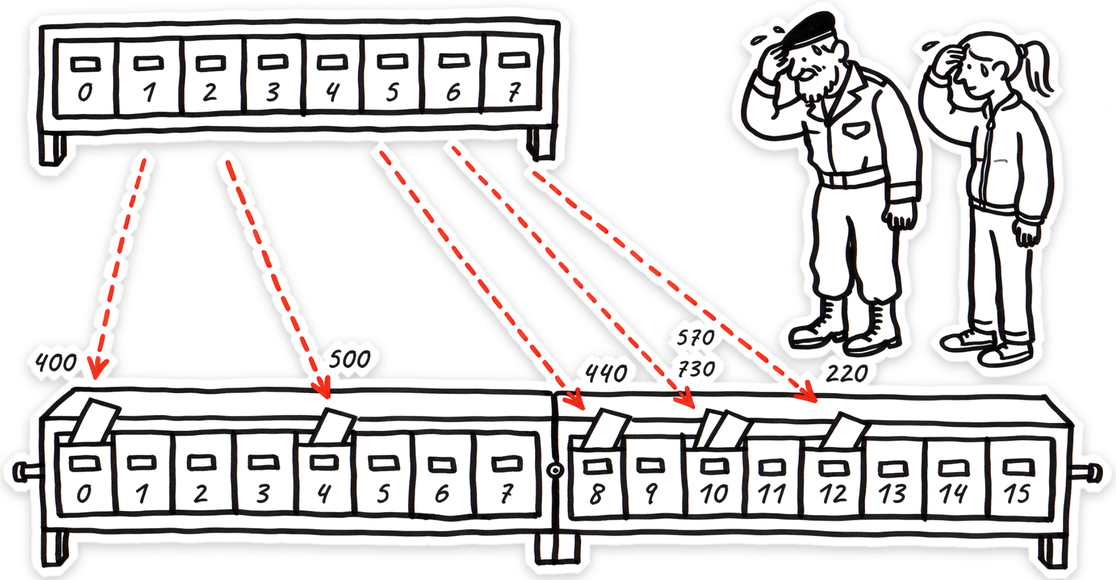
настоящий dict: на 8 ящиков не больше 5 ключей



# Шкаф на 16 ящиков. Кто остался соседями?

```
0 [400]
4 [500]
8 [440]
10 [570, 730]
12 [220]
```

```
old = [[400, 440], [], [570, 730], [], [220, 500], [], [], []]
new = [[] for _ in range(16)]
for drawer in old:
    for app_id in drawer:
        new[hash(app_id) % 16].append(app_id)

for number, drawer in enumerate(new):
    if drawer:
        print(number, drawer)
```

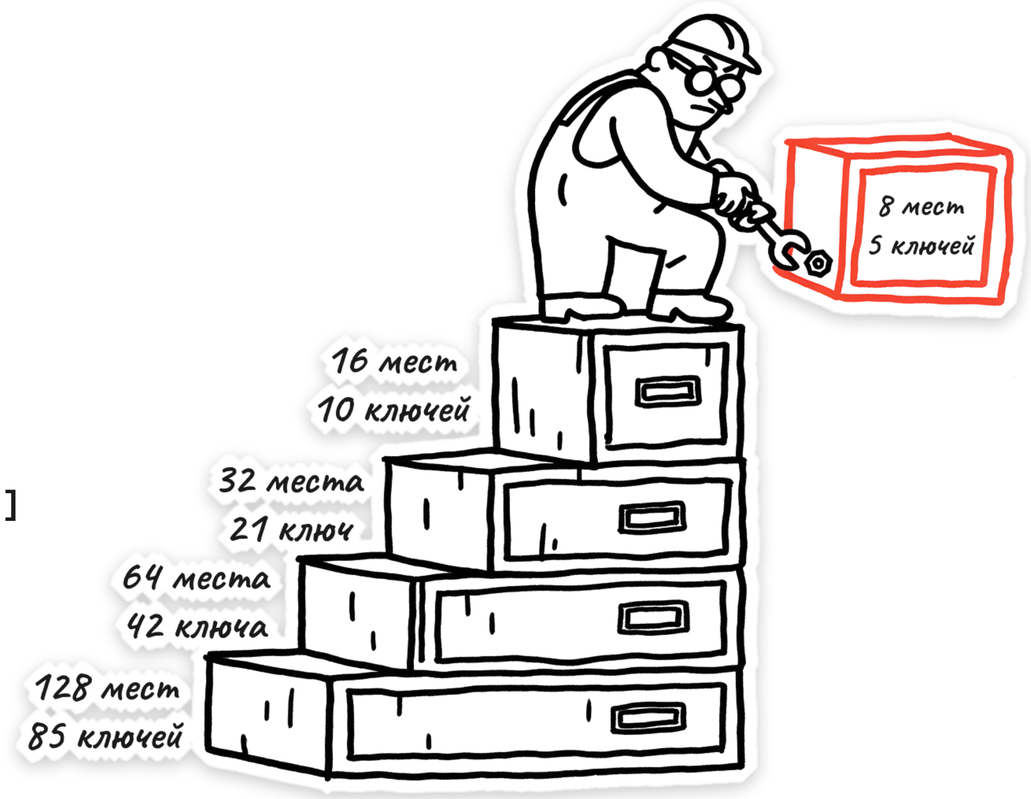


# А на настоящем словаре?

```
import sys

library = {}
jumps = []
for app_id in range(100):
    library[app_id] = "игра"
    size = sys.getsizeof(library)
    if not jumps or jumps[-1][1] != size:
        jumps.append((len(library), size))
print(jumps)
print([places * 2 // 3 + 1 for places in [8, 16, 32, 64, 128]])
```

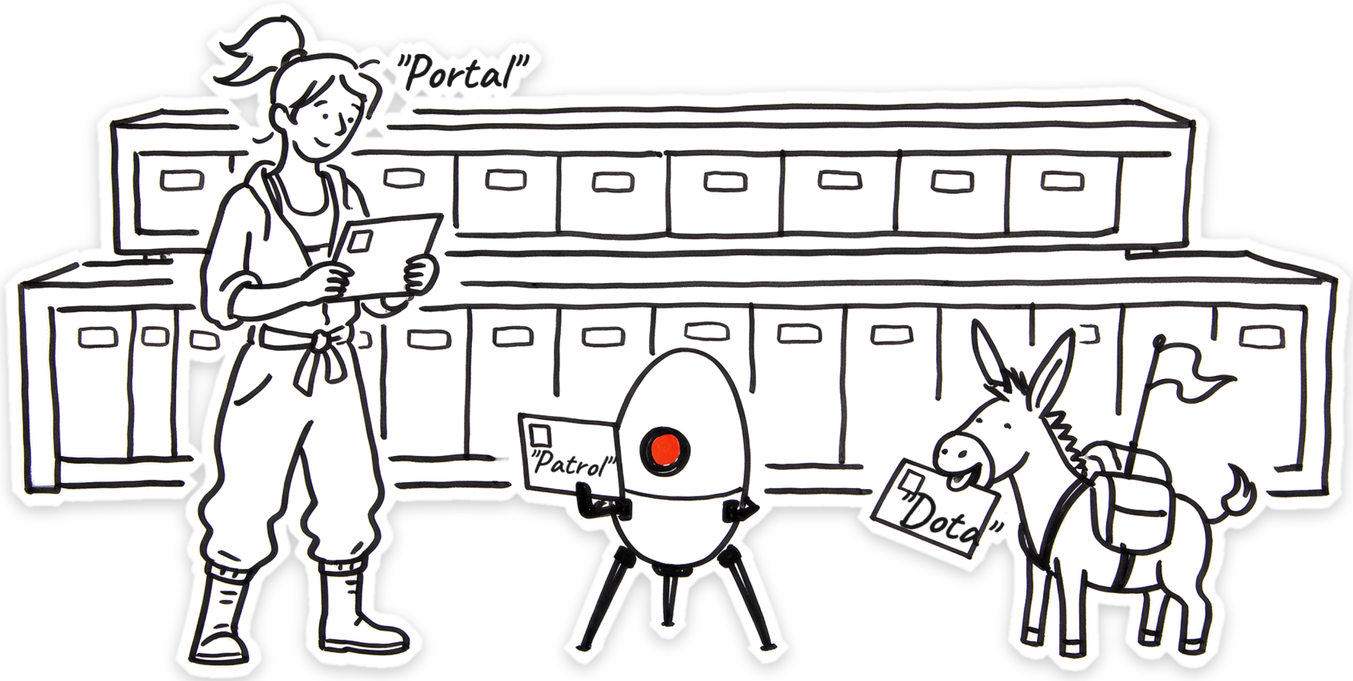
```
[(1, 224), (6, 352), (11, 632), (22, 1168), (43, 2264), (86, 4688)]
[6, 11, 22, 43, 86]
```

ящиков: 8 → 16 → 32 → 64 → 128
влезает ключей: 5 → 10 → 21 → 42 → 85
рост на ключе: 6 → 11 → 22 → 43 → 86



# Ключ — строка. Куда попадёт Patrol?

```
def toy_hash(word):
    return sum(ord(letter) for letter in word)

for title in ["Portal", "Dota", "Patrol"]:
    code = toy_hash(title)
    print(title, code, code % 8, code % 16)
```

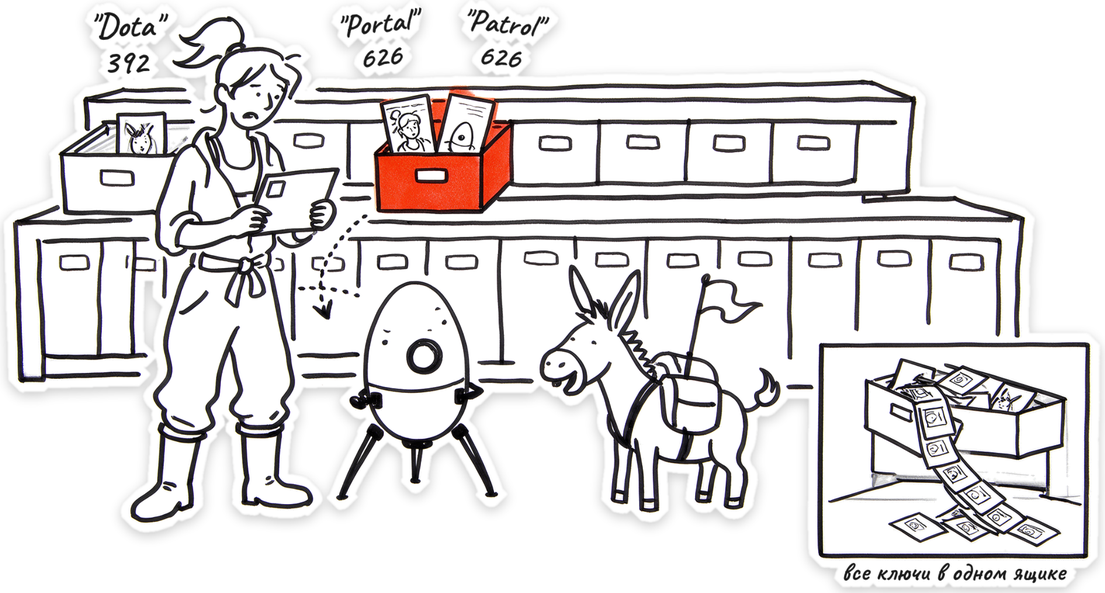


# Равный хеш — один ящик

```
def toy_hash(word):
    return sum(ord(letter) for letter in word)

for title in ["Portal", "Dota", "Patrol"]:
    code = toy_hash(title)
    print(title, code, code % 8, code % 16)
```

```
Portal 626 2 2
Dota 392 0 8
Patrol 626 2 2
```

все ключи в одном ящике → поиск перебором

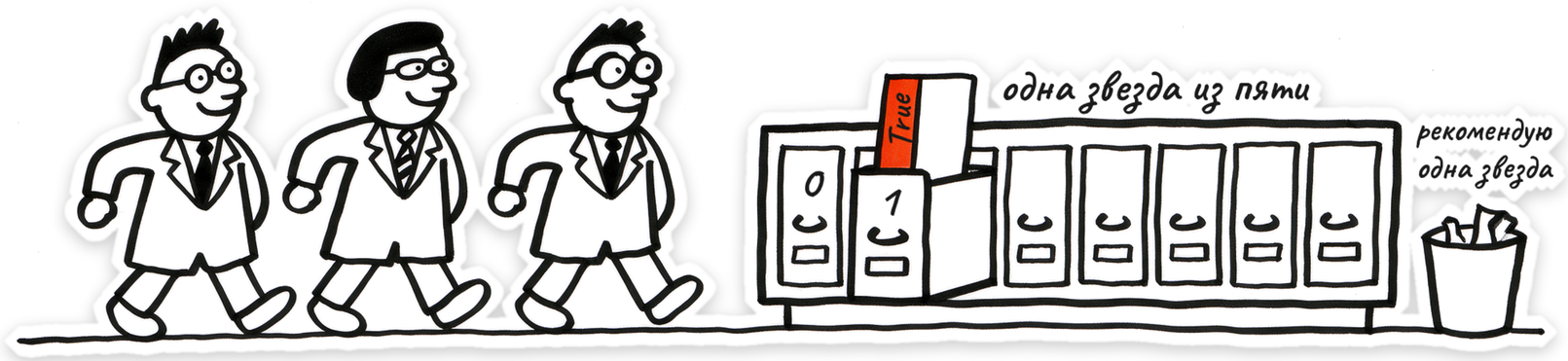


# Возвращаемся к отзывам

```
one, one_float = 1, 1.0
print(one == one_float == True)
print(hash(one), hash(one_float), hash(True))
print(one is one_float)
review = {True: "рекомендую", 1: "одна звезда", 1.0: "одна звезда из пяти"}
print(review)
print(len({1, 1.0, True}))
```

```
True
1 1 1
False
{True: 'одна звезда из пяти'}
1
```

Первый объект ключа остаётся, значение — последнее



# 3
# Правила для ключа

# Два правила для ключа

```
print(hash(-1), hash(-2), -1 == -2)
reviews = {-1: "не рекомендую", -2:
"рекомендую"}
print(len(reviews))
```

```
-2 -2
False
2
```

1. Равные ключи дают равный хеш
2. Хеш ключа не меняется, пока ключ в таблице

Обратное к правилу 1 неверно: равный хеш — ещё не равные ключи

# Где шпион?

```
def toy_hash(words):
    return sum(ord(letter) for word in words for letter in word)

def put(table, key, value):
    table[toy_hash(key) % len(table)].append([key, value])

def get(table, key):
    for pair in table[toy_hash(key) % len(table)]:
        if pair[0] == key:
            return pair[1]
    return None

table = [[] for _ in range(8)]
disguise = ["Engineer", "RED"]
put(table, disguise, "Spy")
print(toy_hash(disguise) % 8, get(table, ["Engineer", "RED"]))

disguise[0] = "Medic"
print(toy_hash(disguise) % 8, get(table, ["Medic", "RED"]))
print(get(table, ["Engineer", "RED"]))
print(table[0])
```

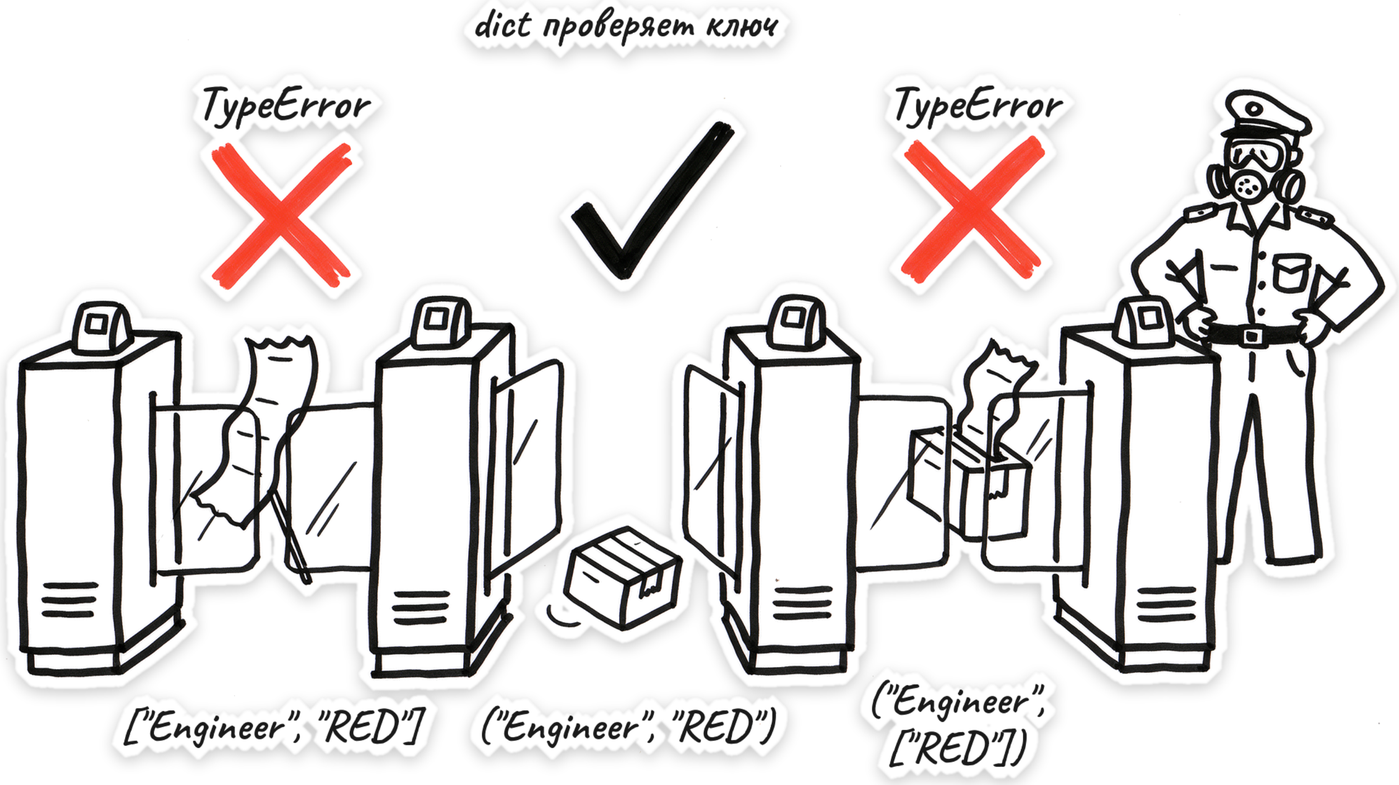
# Запись есть, найти нельзя

```
def toy_hash(words):
    return sum(ord(letter) for word in words for letter in word)

def put(table, key, value):
    table[toy_hash(key) % len(table)].append([key, value])

def get(table, key):
    for pair in table[toy_hash(key) % len(table)]:
        if pair[0] == key:
            return pair[1]
    return None

table = [[] for _ in range(8)]
disguise = ["Engineer", "RED"]
put(table, disguise, "Spy")
print(toy_hash(disguise) % 8, get(table, ["Engineer", "RED"]))

disguise[0] = "Medic"
print(toy_hash(disguise) % 8, get(table, ["Medic", "RED"]))
print(get(table, ["Engineer", "RED"]))
print(table[0])
```

```
0 Spy
5 None
None
[[['Medic', 'RED'], 'Spy']]
```

# Настоящий словарь не даёт так сделать

```
spies = {}
try:
    spies[["Engineer", "RED"]] = "Spy"
except TypeError as error:
    print("TypeError:", error)

spies[("Engineer", "RED")] = "Spy"
print(spies)
try:
    spies[("Engineer", ["RED"])] = "Spy"
except TypeError as error:
    print("TypeError:", error)
```




```
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
{('Engineer', 'RED'): 'Spy'}
TypeError: cannot use 'tuple' as a dict key (unhashable type: 'list')
```

# Что может быть ключом?

```
candidates = [
    730,
    "Portal 2",
    ("AK-47", "Redline", "Field-Tested"),
    ["Bill", "Zoey"],
    frozenset({"Bill", "Zoey"}),
    ("Coach", ["Ellis"]),
    {"app_id": 220},
    None,
]
for number, key in enumerate(candidates, 1):
    try:
        hash(key)
        print(number, "да")
    except TypeError:
        print(number, "нет")
```

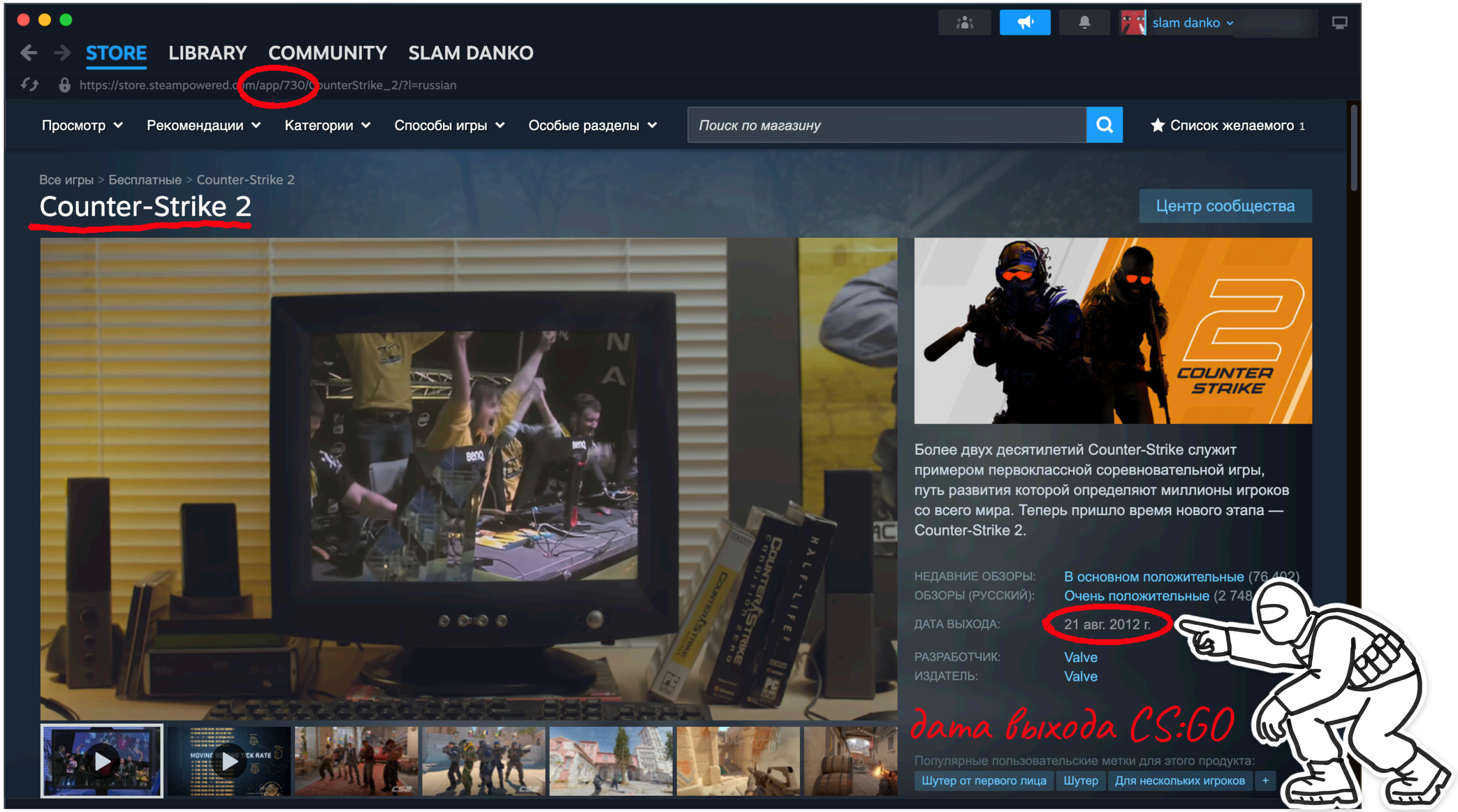
# Ключ неизменяем целиком

```
candidates = [
    730,
    "Portal 2",
    ("AK-47", "Redline", "Field-Tested"),
    ["Bill", "Zoey"],
    frozenset({"Bill", "Zoey"}),
    ("Coach", ["Ellis"]),
    {"app_id": 220},
    None,
]
for number, key in enumerate(candidates, 1):
    try:
        hash(key)
        print(number, "да")
    except TypeError:
        print(number, "нет")
```

```
1 да
2 да
3 да
4 нет
5 да
6 нет
7 нет
8 да
```

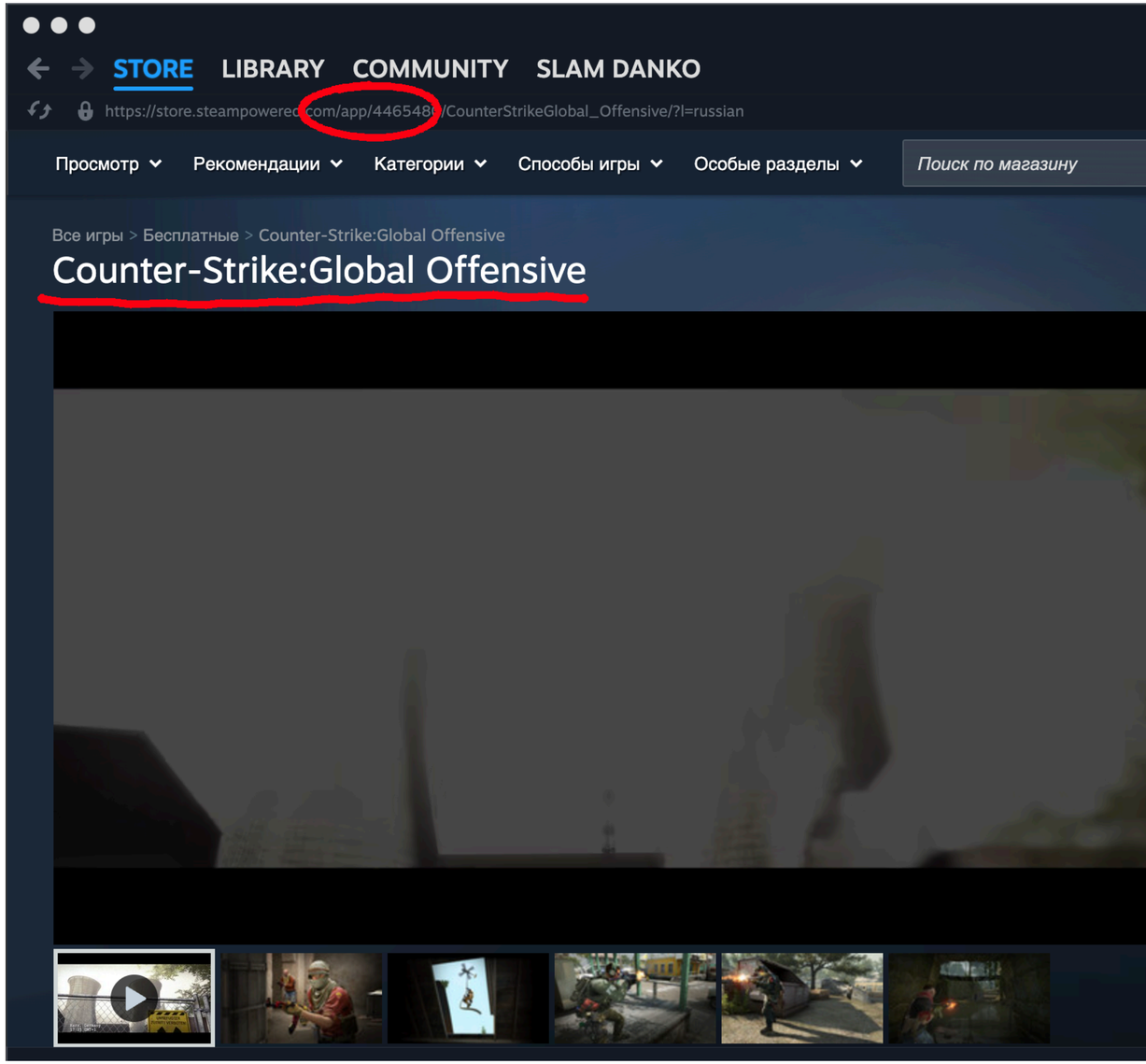
# 4
# Выбор ключа

# Одна игра, два названия



# Что найдётся по названию?

```
store = {730: "Counter-Strike: Global Offensive"}
reviews = {"Counter-Strike: Global Offensive": ["отличная игра"]}

store[730] = "Counter-Strike 2"
store[4465480] = "Counter-Strike:Global Offensive"

print(store)
print(reviews.get(store[730]))
```

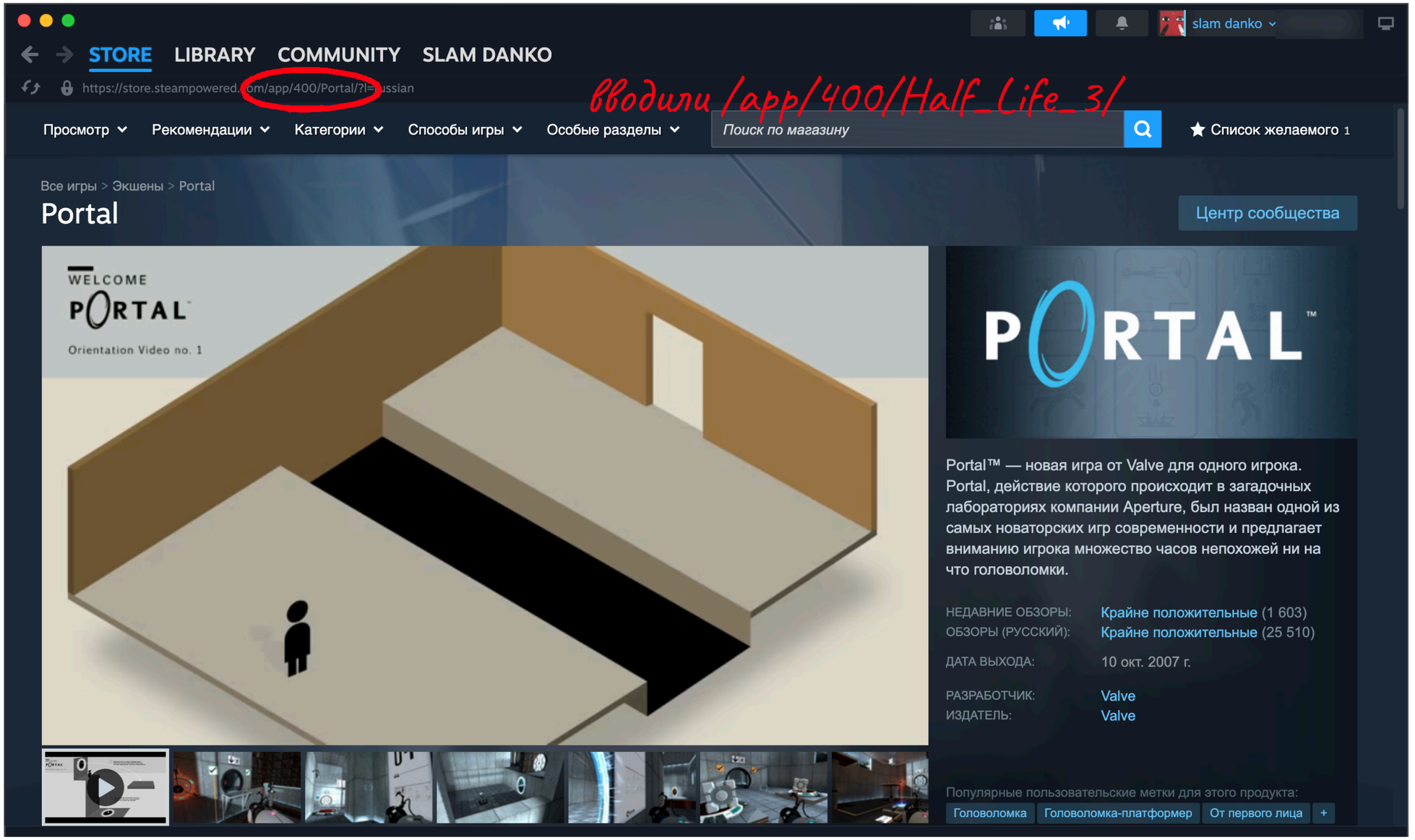


# Название — значение, app id — ключ

```
store = {730: "Counter-Strike: Global Offensive"}
reviews = {"Counter-Strike: Global Offensive": ["отличная игра"]}

store[730] = "Counter-Strike 2"
store[4465480] = "Counter-Strike:Global Offensive"

print(store)
print(reviews.get(store[730]))
```

```
{730: 'Counter-Strike 2', 4465480: 'Counter-Strike:Global Offensive'}
None
```

# Название в адресе — только подпись



# Ник меняется, SteamID — нет

ИМЯ ПРОФИЛЯ

slam danko

меняется когда угодно

НАСТОЯЩЕЕ ИМЯ ?

ЛИЧНАЯ ССЫЛКА ?

noidandroid

тоже меняется

Ваш профиль будет находиться по адресу: https://steamcommunity.com/id/no

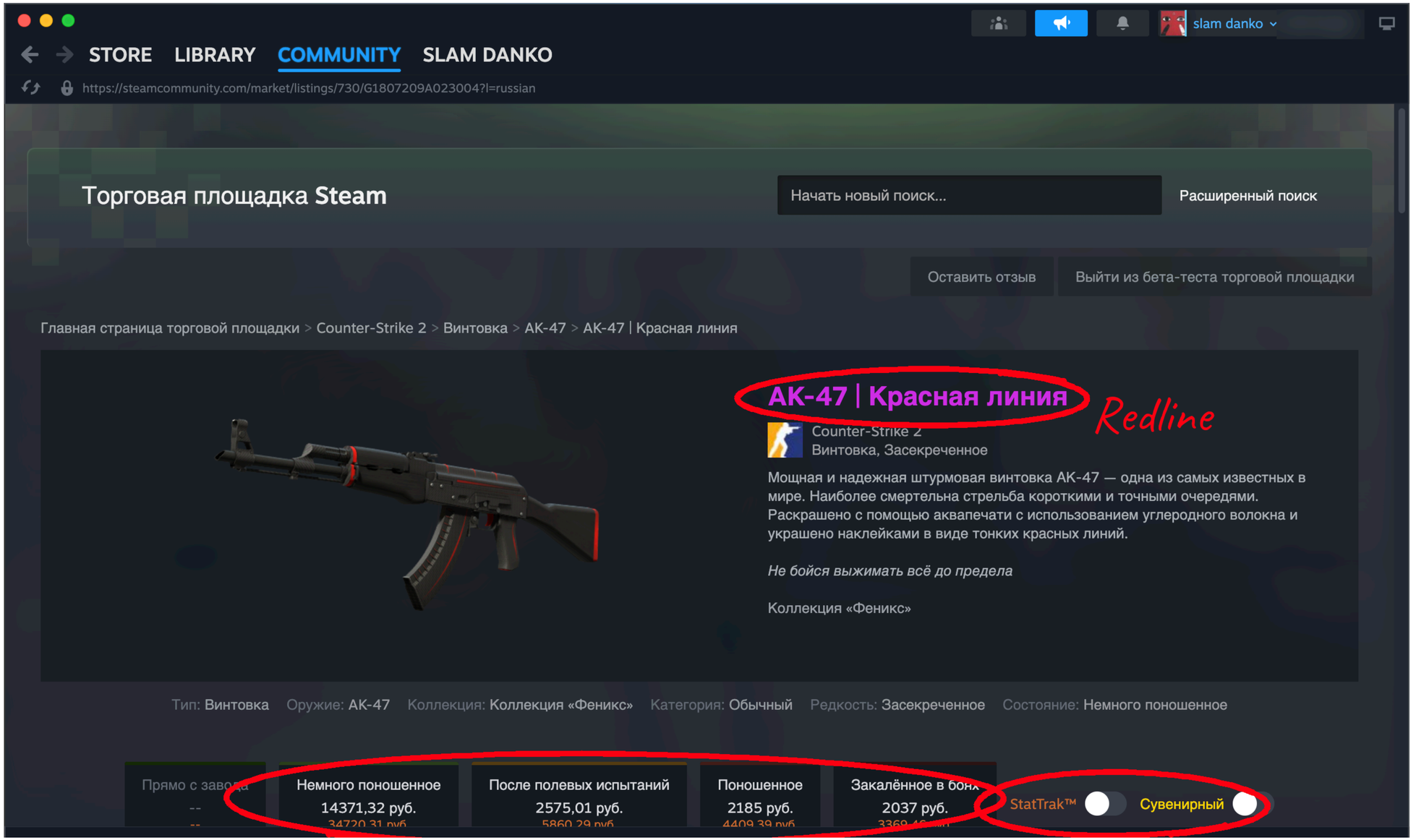
Главная > Аккаунт

Аккаунт пользователя

Steam ID 76561198125736486

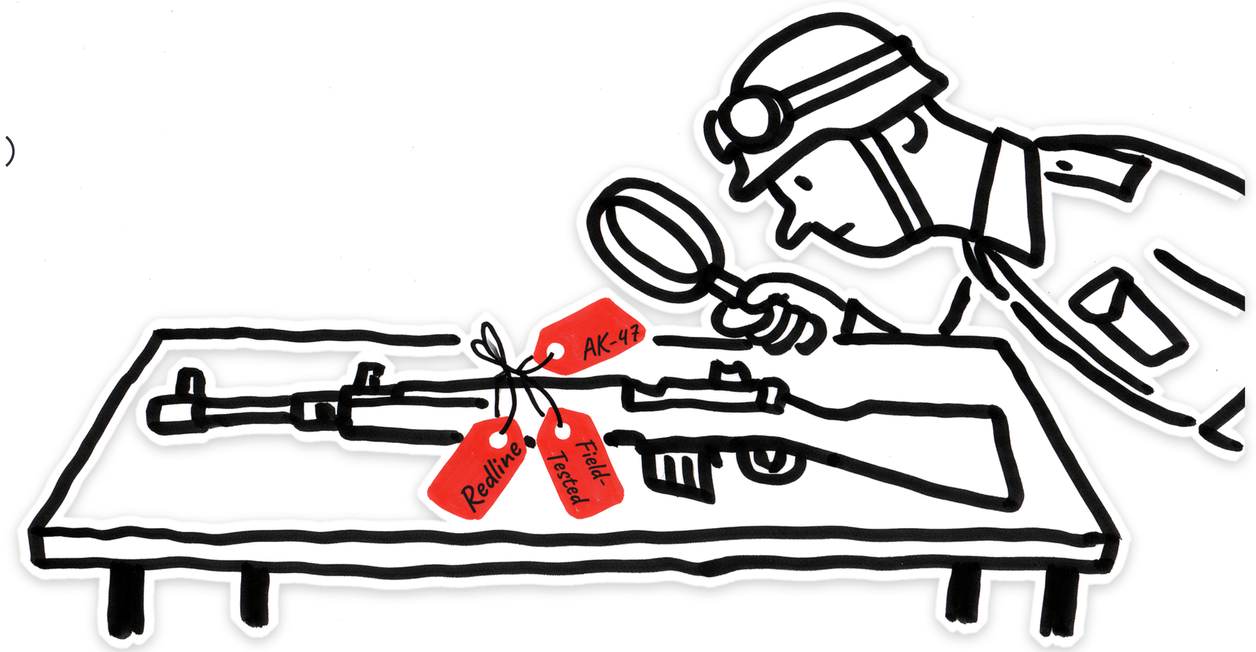
не меняется

# Позиция на торговой площадке



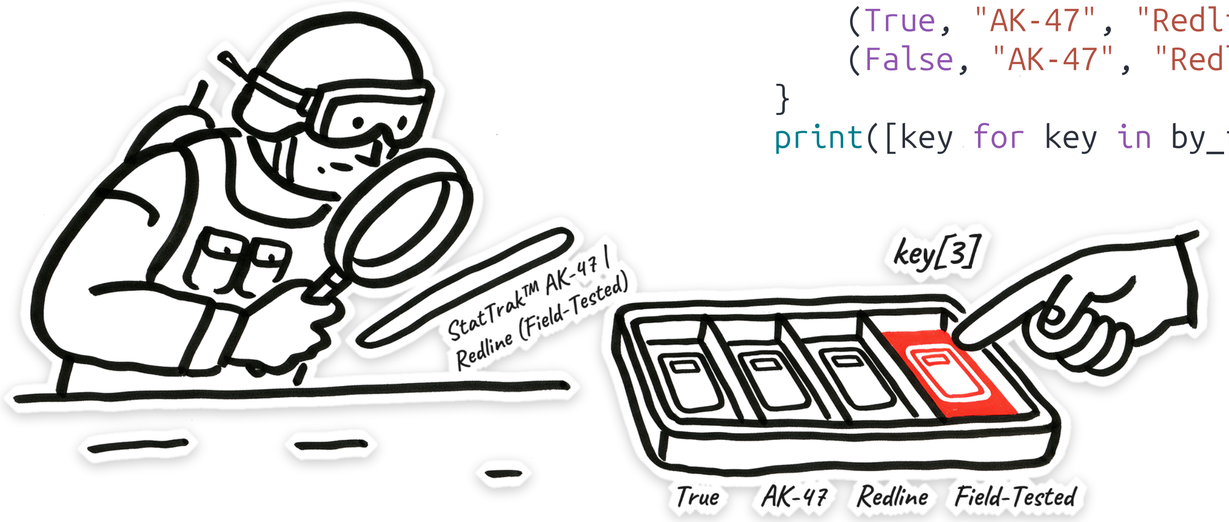
# Найдите все Field-Tested

```
listings = {  # цены в $ на 24.10.2026
    "AK-47 | Redline (Field-Tested)": 31.99,
    "StatTrak™ AK-47 | Redline (Field-Tested)": 72.79,
    "AK-47 | Redline (Minimal Wear)": 178.50,
}
field_tested = [name for name in listings if name.endswith("(Field-Tested)")]
print(field_tested)

by_tuple = {
    (False, "AK-47", "Redline", "Field-Tested"): 31.99,
    (True, "AK-47", "Redline", "Field-Tested"): 72.79,
    (False, "AK-47", "Redline", "Minimal Wear"): 178.50,
}
print([key for key in by_tuple if key[3] == "Field-Tested"])
```



# Склейку приходится разбирать

```python
listings = {  # цены в $ на 24.10.2026
    "AK-47 | Redline (Field-Tested)": 31.99,
    "StatTrak™ AK-47 | Redline (Field-Tested)": 72.79,
    "AK-47 | Redline (Minimal Wear)": 178.50,
}
field_tested = [name for name in listings if name.endswith("(Field-Tested)")]
print(field_tested)

by_tuple = {
    (False, "AK-47", "Redline", "Field-Tested"): 31.99,
    (True, "AK-47", "Redline", "Field-Tested"): 72.79,
    (False, "AK-47", "Redline", "Minimal Wear"): 178.50,
}
print([key for key in by_tuple if key[3] == "Field-Tested"])
```




```
['AK-47 | Redline (Field-Tested)', 'StatTrak™ AK-47 | Redline (Field-Tested)']
[(False, 'AK-47', 'Redline', 'Field-Tested'), (True, 'AK-47', 'Redline', 'Field-Tested')]
```

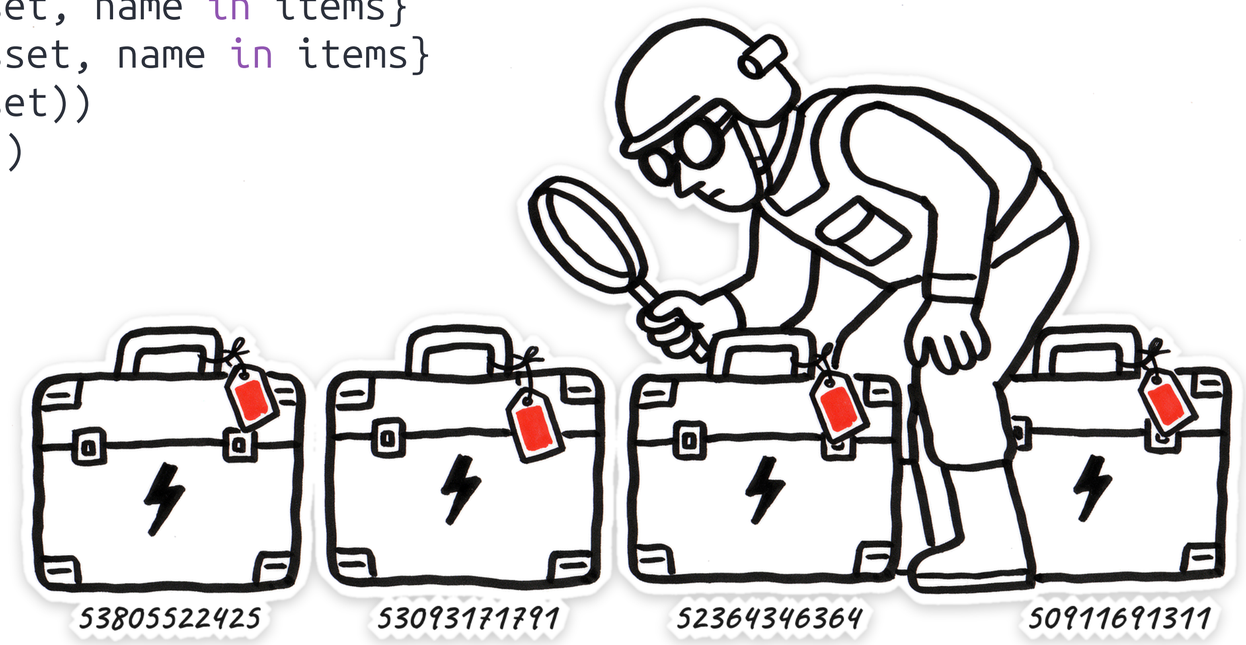
# Четыре одинаковых кейса

```
items = [
    ("53805522425", "Kilowatt Case"),
    ("53093171791", "Kilowatt Case"),
    ("52364346364", "Kilowatt Case"),
    ("50911691311", "Kilowatt Case"),
    ("52694160310", "Revolution Case"),
]
by_name = {name: asset for asset, name in items}
by_asset = {asset: name for asset, name in items}
print(len(by_name), len(by_asset))
print(by_name["Kilowatt Case"])
```

# Одинаковые предметы — разные ключи

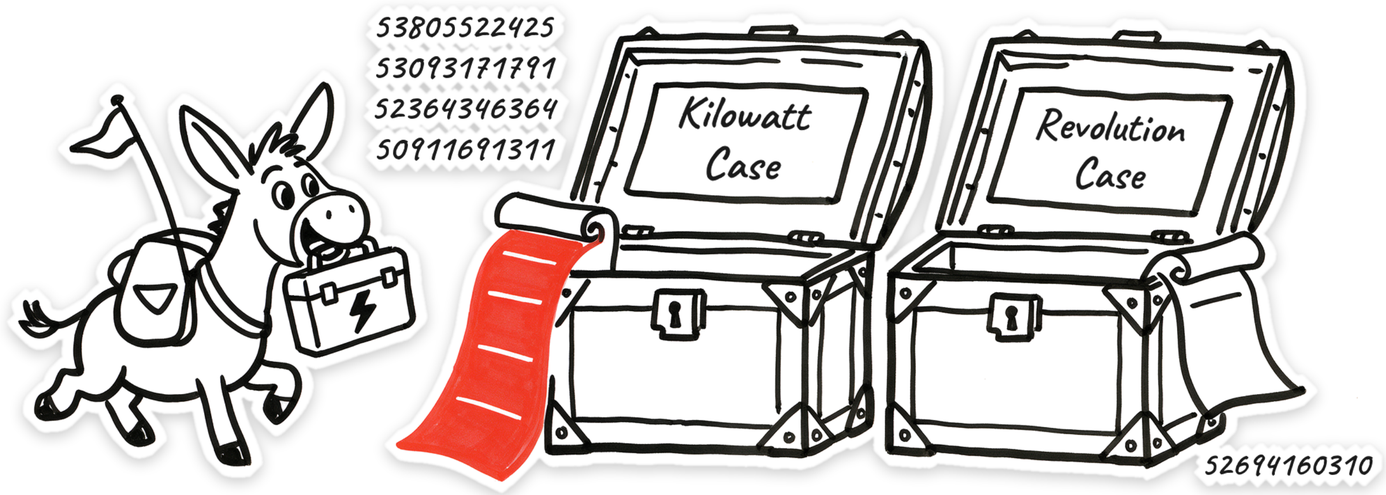
```
items = [
    ("53805522425", "Kilowatt Case"),
    ("53093171791", "Kilowatt Case"),
    ("52364346364", "Kilowatt Case"),
    ("50911691311", "Kilowatt Case"),
    ("52694160310", "Revolution Case"),
]
by_name = {name: asset for asset, name in items}
by_asset = {asset: name for asset, name in items}
print(len(by_name), len(by_asset))
print(by_name["Kilowatt Case"])
```

```
2 5
50911691311
```



# Название → список номеров

```
items = [
    ("53805522425", "Kilowatt Case"),
    ("53093171791", "Kilowatt Case"),
    ("52364346364", "Kilowatt Case"),
    ("50911691311", "Kilowatt Case"),
    ("52694160310", "Revolution Case"),
]
cases = {}
for asset, name in items:
    if name not in cases:
        cases[name] = []
    cases[name].append(asset)
print(len(cases))
print(cases["Kilowatt Case"])
```




```
2
['53805522425', '53093171791', '52364346364', '50911691311']
```

ключ — название, значение — список

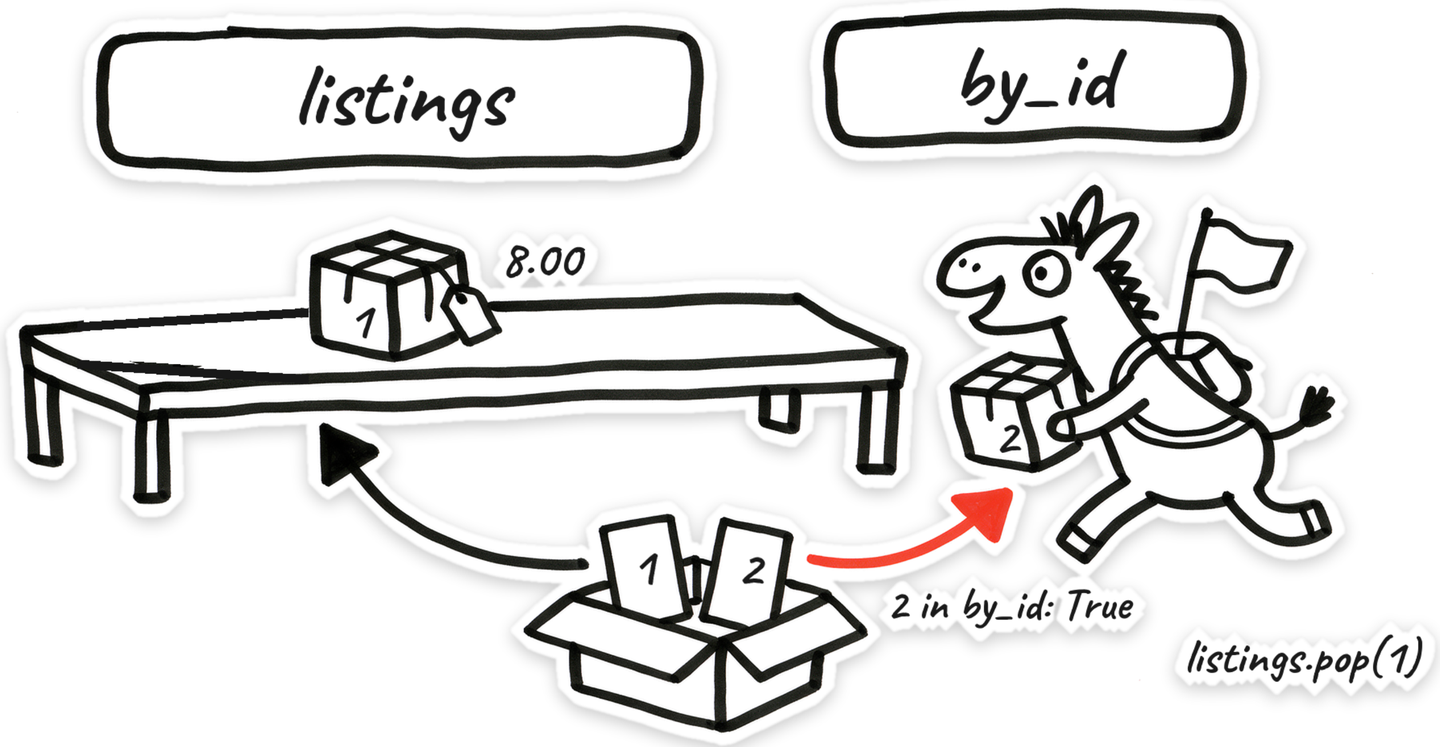
# Индекс по номеру лота

```
listings = [
    {"id": 1, "item": "Inscribed Arms of Desolation", "price": 9.29},
    {"id": 2, "item": "Inscribed Magus Apex", "price": 2.68},
]
by_id = {listing["id"]: listing for listing in listings}

listings[0]["price"] = 8.00
print(by_id[1]["price"])

listings.pop(1)
print(len(listings), len(by_id), 2 in by_id)
```

# Индекс — второй путь к тем же лотам

```
listings = [
    {"id": 1, "item": "Inscribed Arms of Desolation", "price": 9.29},
    {"id": 2, "item": "Inscribed Magus Apex", "price": 2.68},
]
by_id = {listing["id"]: listing for listing in listings}

listings[0]["price"] = 8.00
print(by_id[1]["price"])

listings.pop(1)
print(len(listings), len(by_id), 2 in by_id)
```

```
8.0
1 2 True
```



# Снимите лот дважды

```
listings = [
    {"id": 1, "item": "Inscribed Arms of Desolation", "price": 9.29},
    {"id": 2, "item": "Inscribed Magus Apex", "price": 2.68},
]
by_id = {listing["id"]: listing for listing in listings}

def cancel(lot_id):
    listings.remove(by_id[lot_id])

cancel(2)
print(len(listings), len(by_id), 2 in by_id)
cancel(2)
```

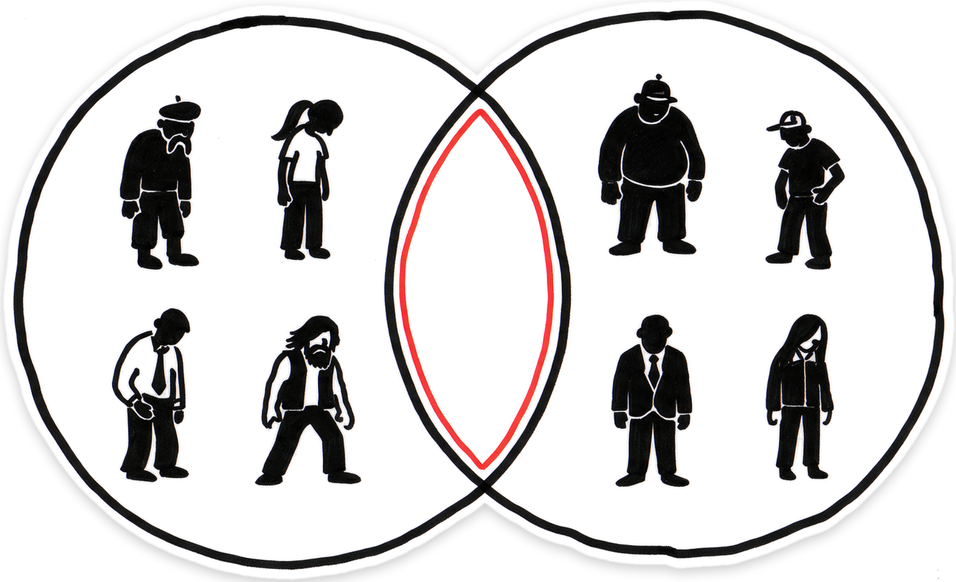
# Сначала проверка, потом обе структуры

```
listings = [
    {"id": 1, "item": "Inscribed Arms of Desolation", "price": 9.29},
    {"id": 2, "item": "Inscribed Magus Apex", "price": 2.68},
]
by_id = {listing["id"]: listing for listing in listings}

def cancel(lot_id):
    if lot_id not in by_id:
        return False
    lot = by_id.pop(lot_id)
    listings.remove(lot)
    return True

print(cancel(2), cancel(2))
print(len(listings), len(by_id), 2 in by_id)
```

```
True False
1 1 False
```

было: 1 2 True
ValueError: list.remove(x): x not in list

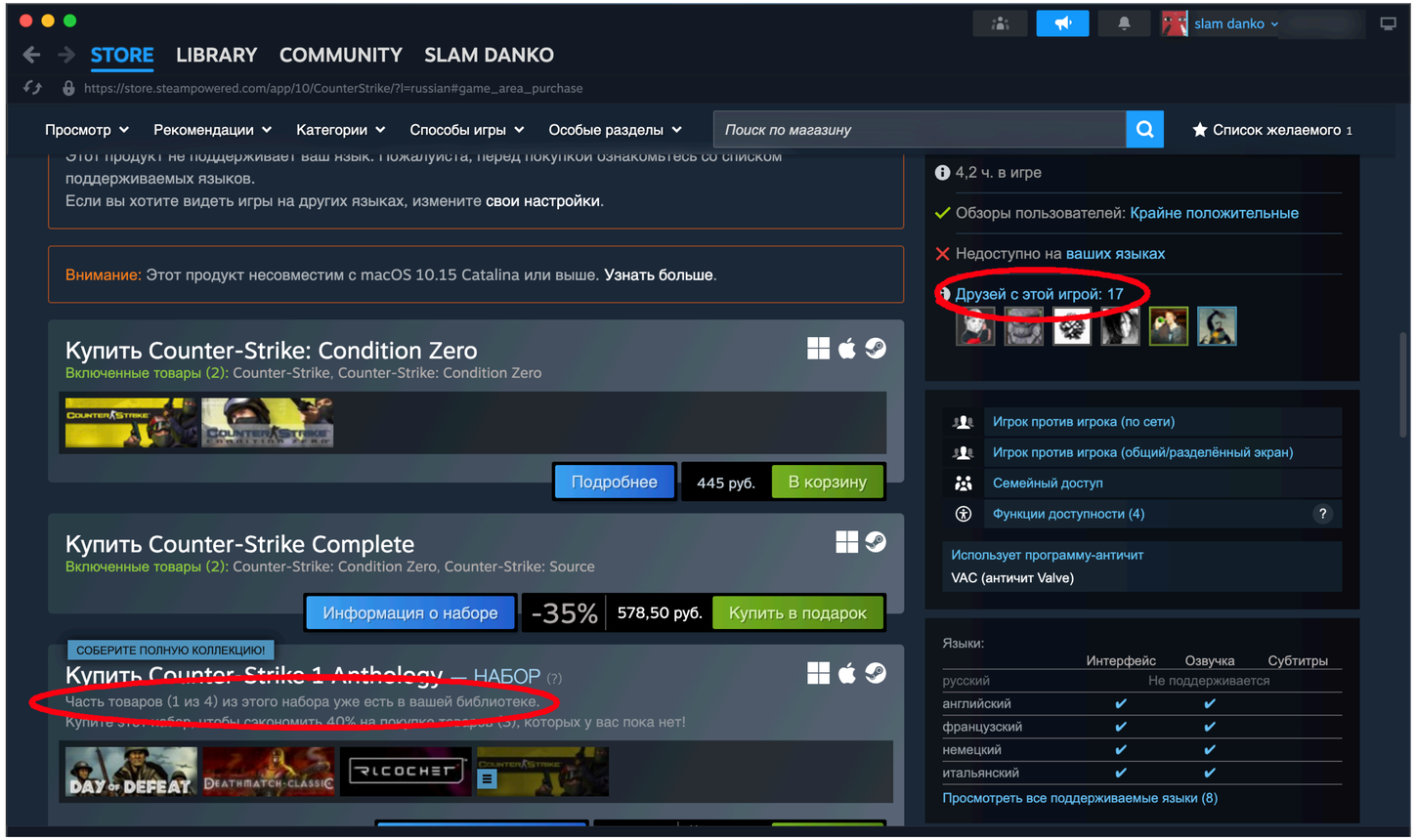
# 5
# Множества и пустые ответы



# Во что сыграть с другом?

```
mine = {220, 400, 440, 620, 730}
friend = {400, 440, 550, 570, 620}
wishlist = {546560, 550, 400}

print(sorted(mine & friend))
print(sorted(mine | friend))
print(sorted(wishlist - mine))
print(sorted(mine ^ friend))
```

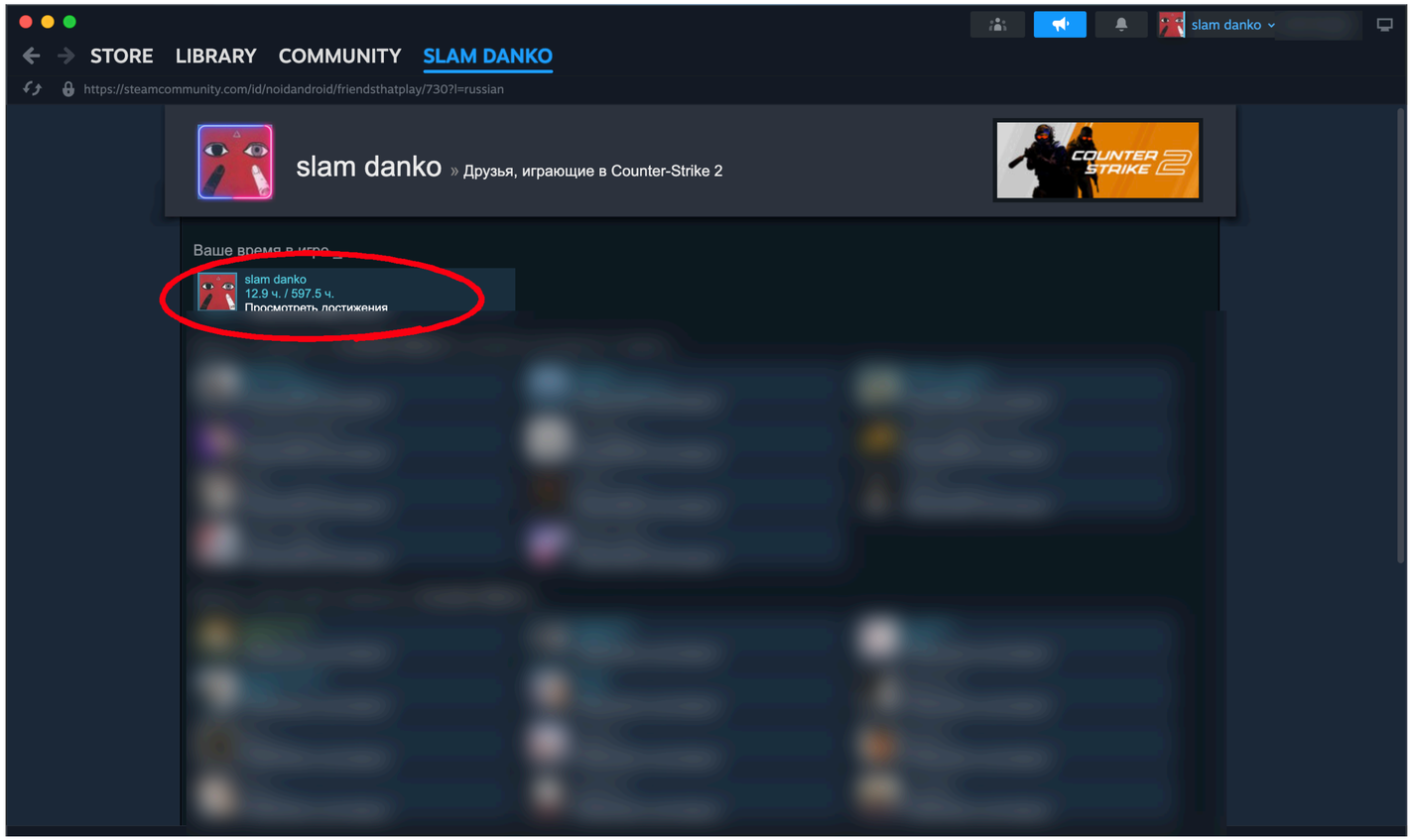
& есть у обоих
| есть хотя бы у одного
- есть только в первом
^ есть ровно у одного



# Четыре операции над множествами

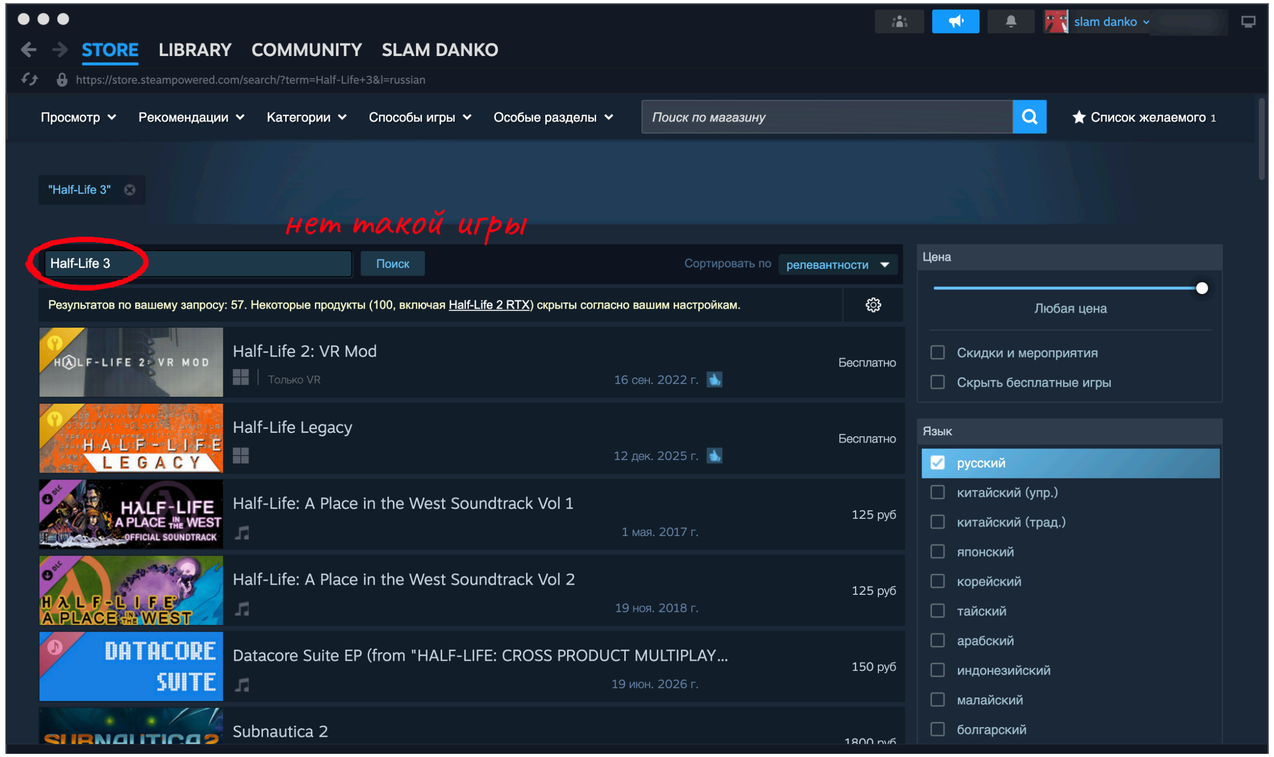
```
mine = {220, 400, 440, 620, 730}
friend = {400, 440, 550, 570, 620}
wishlist = {546560, 550, 400}

print(sorted(mine & friend))
print(sorted(mine | friend))
print(sorted(wishlist - mine))
print(sorted(mine ^ friend))
```

```
[400, 440, 620]
[220, 400, 440, 550, 570, 620, 730]
[550, 546560]
[220, 550, 570, 730]
```



# Гейб, где Half-Life 3?

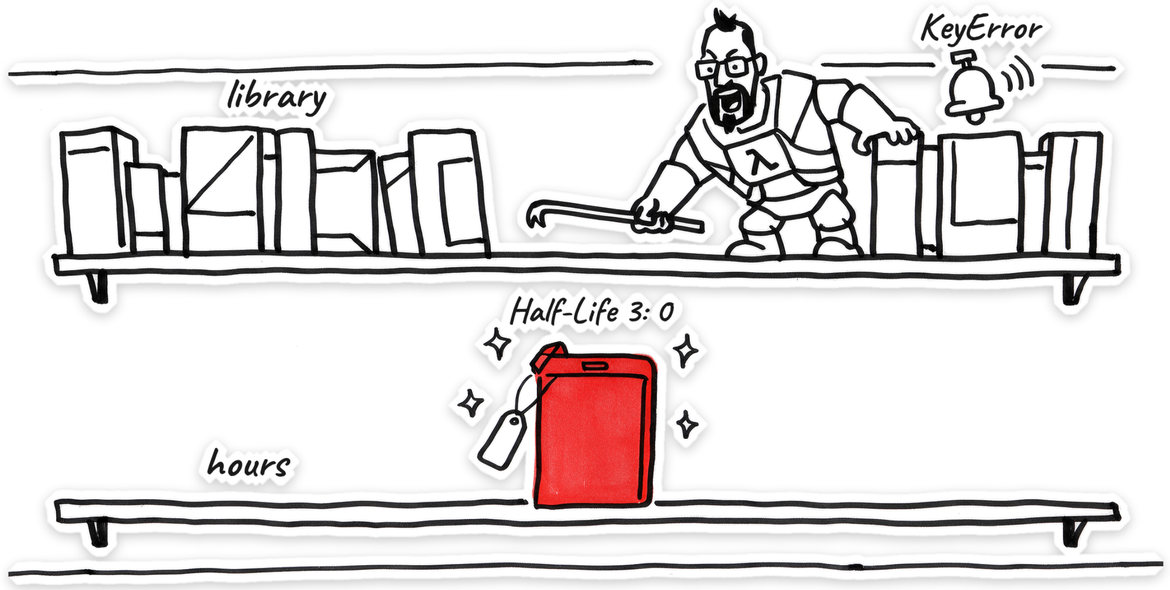
```
from collections import defaultdict

library = {"Half-Life": 12, "Half-Life 2": 30}
print(library.get("Half-Life 3"))

hours = defaultdict(int)
print(hours.get("Half-Life 3"), "Half-Life 3" in hours)
print(hours["Half-Life 3"], "Half-Life 3" in hours)
try:
    library["Half-Life 3"]
except KeyError as error:
    print("KeyError:", error)
```



# defaultdict создал Half-Life 3

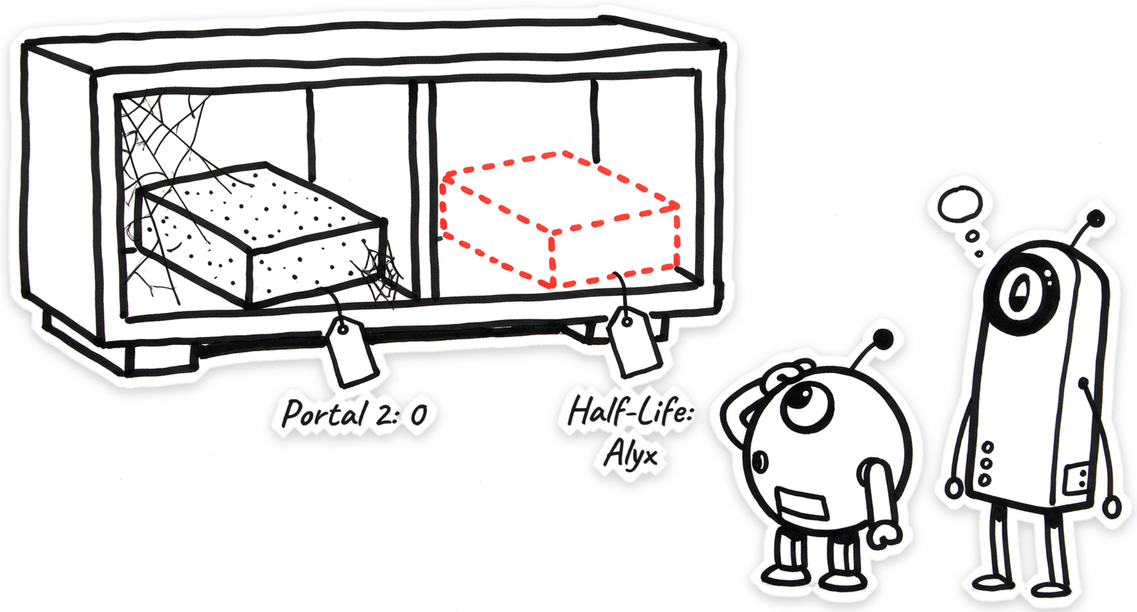
```python
from collections import defaultdict

library = {"Half-Life": 12, "Half-Life 2": 30}
print(library.get("Half-Life 3"))

hours = defaultdict(int)
print(hours.get("Half-Life 3"), "Half-Life 3" in hours)
print(hours["Half-Life 3"], "Half-Life 3" in hours)
try:
    library["Half-Life 3"]
except KeyError as error:
    print("KeyError:", error)
```

**None**
**None False**
**0 True**
**KeyError: 'Half-Life 3'**



# Ноль часов

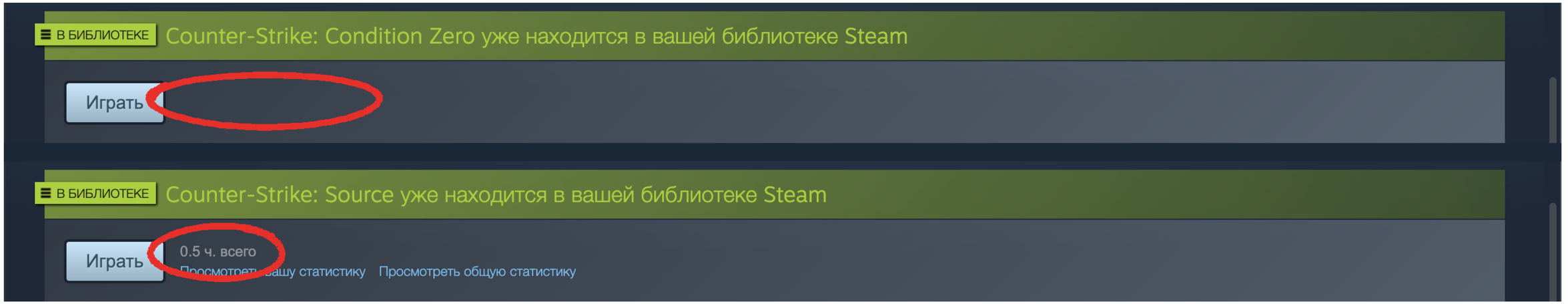
```
hours = {"Portal 2": 0, "Dota 2": 1200}
for title in ["Portal 2", "Half-Life: Alyx"]:
    if hours.get(title):
        print(title, "играл")
    else:
        print(title, "не играл?")
print("Portal 2" in hours, "Half-Life: Alyx" in hours)
```



# Ноль и «нет» — разные ответы

```
hours = {"Portal 2": 0, "Dota 2": 1200}
for title in ["Portal 2", "Half-Life: Alyx"]:
    if hours.get(title):
        print(title, "играл")
    else:
        print(title, "не играл?")
print("Portal 2" in hours, "Half-Life: Alyx" in hours)
```

```
Portal 2 не играл?
Half-Life: Alyx не играл?
True False
```



# Как Steam хранит часы

```
"apps"
{
    "240"
    {
        "LastPlayed"    "1425052068"
        "Playtime"      "30"
    }
    "440"
    {
        "LastPlayed"    "1632853696"
        "Playtime"      "53"
    }
}
```

ключи — строки с app id
минуты: 30 = 0.5 ч
Condition Zero (80) здесь нет

# Словарь помнит порядок, множество — нет

```
recent = {"Portal": 1, "Dota 2": 2, "Half-Life 2": 3}
recent["Portal"] = 4
print(list(recent))
print(list({"Portal", "Dota 2", "Half-Life 2", "Left 4 Dead 2"}))
```

```
['Portal', 'Dota 2', 'Half-Life 2']
['Portal', 'Left 4 Dead 2', 'Dota 2', 'Half-Life 2']
```

Словарь помнит порядок вставки
Множество порядок не хранит

# Выводы

> dict и set — хеш-таблицы: hash выбирает ящик, == находит ключ

> Таблица растёт заранее: поиск в среднем не зависит от размера

> Равные ключи → равный хеш, но равный хеш — ещё не равные ключи

> Ключ хешируем и стабилен: app id, а не название

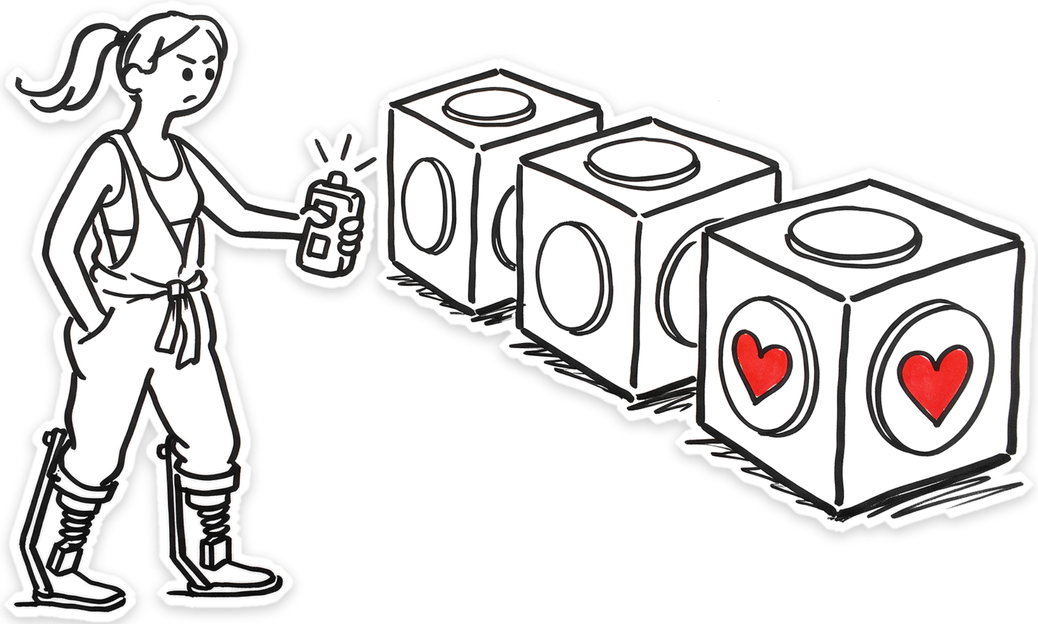
> Индекс меняем вместе с данными, перед удалением проверяем ключ

> Есть ли ключ — проверяем через in, а не через if d.get(k)

# Резерв (бонусные материалы)

# Сколько кубов в множестве?

```
def make_cube(*marks):
    return tuple(["кубик", "50 кг", *marks])

storage = make_cube()
other = make_cube()
companion = make_cube("сердечко")

print(storage == other, storage is other)
print(len({storage, other}))
print(len({storage, other, companion}))
```

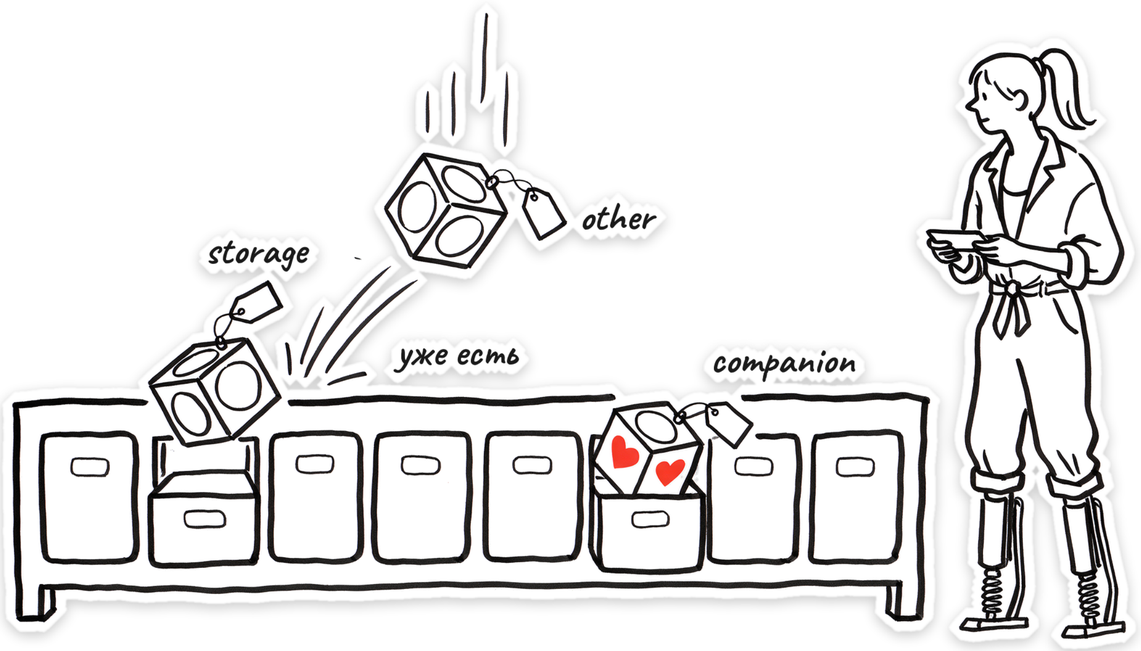


# Равные кубы — один элемент

```
def make_cube(*marks):
    return tuple(["кубик", "50 кг", *marks])

storage = make_cube()
other = make_cube()
companion = make_cube("сердечко")

print(storage == other, storage is other)
print(len({storage, other}))
print(len({storage, other, companion}))
```

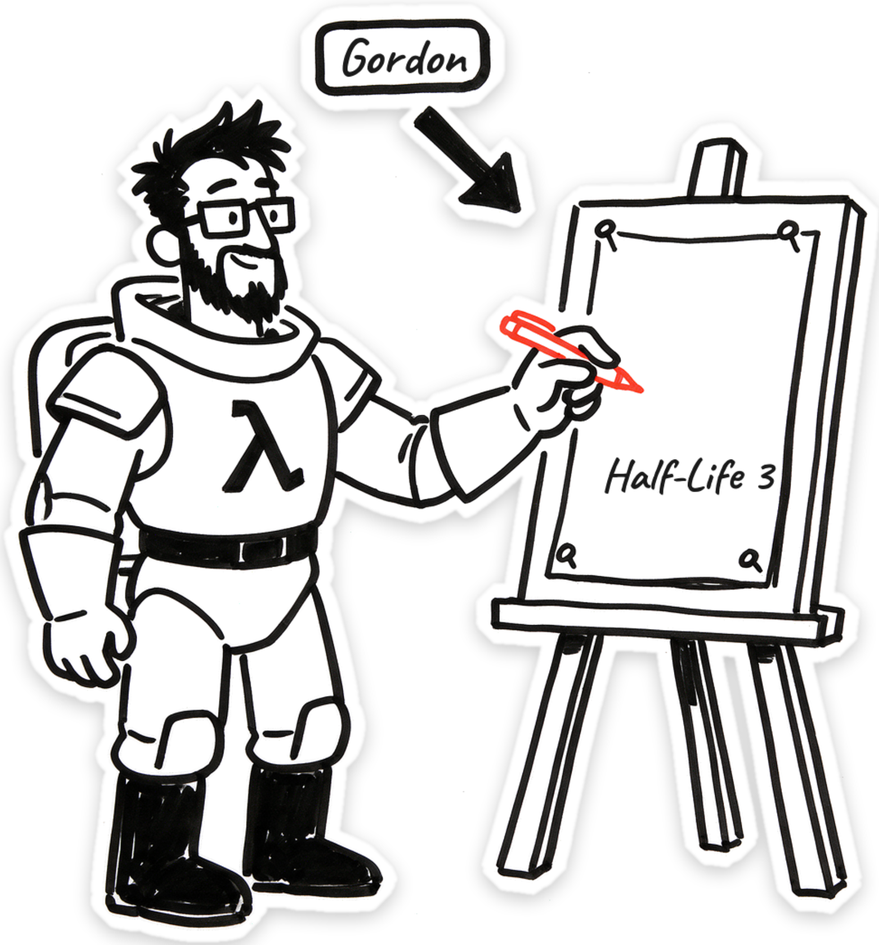
```
True False
1
2
```

set — хеш-таблица без значений



# Список желаемого для каждого друга

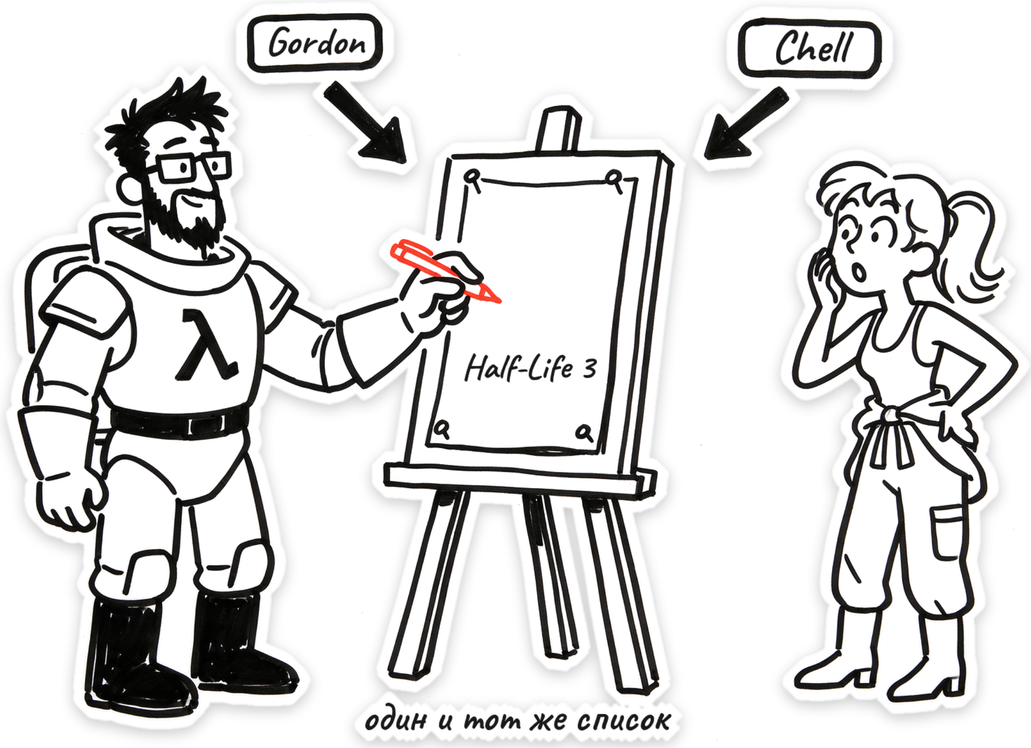
```
wishlists = dict.fromkeys(["Gordon", "Chell"], [])
wishlists["Gordon"].append("Half-Life 3")
print(wishlists)
```



# Один список на двоих

```
wishlists = dict.fromkeys(["Gordon", "Chell"], [])
wishlists["Gordon"].append("Half-Life 3")
print(wishlists)
```

```
{'Gordon': ['Half-Life 3'], 'Chell': ['Half-Life 3']}
```



# А если поменять порядок имён?

```
from functools import lru_cache

@lru_cache
def price(app_id, region):
    return 9.99

price(app_id=400, region="ru")
price(region="ru", app_id=400)
print(price.cache_info().misses)
```

# Два разных ключа

```
from functools import lru_cache

@lru_cache
def price(app_id, region):
    return 9.99

price(app_id=400, region="ru")
price(region="ru", app_id=400)
print(price.cache_info().misses)
```

**2**

Порядок именованных аргументов входит в ключ

# Глобальные имена лежат в словаре

globals() → dict
sys.modules → dict
модуль.__dict__ → dict

# Хеш строки меняется от запуска к запуску

```
$ PYTHONHASHSEED=1 python3 -c 'print(hash(400), hash("Portal") % 8)'
400 3
$ PYTHONHASHSEED=2 python3 -c 'print(hash(400), hash("Portal") % 8)'
400 6
$ PYTHONHASHSEED=3 python3 -c 'print(hash(400), hash("Portal") % 8)'
400 1
```

# Сколько ключей в самом длинном ящике?

```
def longest_bucket(keys, size):
    table = [[] for _ in range(size)]
    for key in keys:
        table[hash(key) % size].append(key)
    return max(len(bucket) for bucket in table)

library = range(200_000)
for size in [1, 8, 1024, 262_144]:
    print(size, longest_bucket(library, size))
print(10, longest_bucket(range(0, 2_000_000, 10), 10))
```

# Короткие ящики — быстрый поиск

```
def longest_bucket(keys, size):
    table = [[] for _ in range(size)]
    for key in keys:
        table[hash(key) % size].append(key)
    return max(len(bucket) for bucket in table)

library = range(200_000)
for size in [1, 8, 1024, 262_144]:
    print(size, longest_bucket(library, size))
print(10, longest_bucket(range(0, 2_000_000, 10), 10))
```

```
1 200000
8 25000
1024 196
262144 1
10 200000
```

заполненность = ключей / ящиков

# Что чаще всего есть у друзей

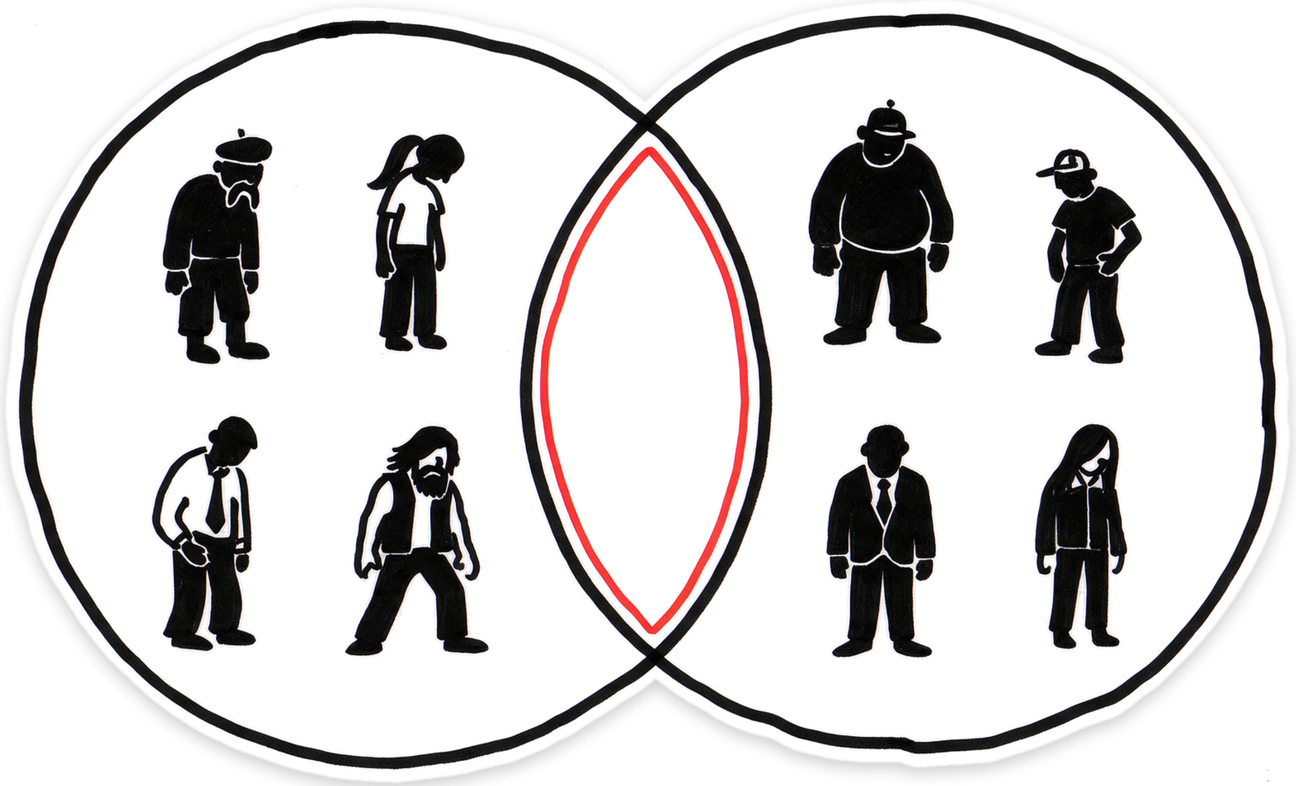
```
from collections import Counter

friends = [
    {"Dota 2", "Portal 2"},
    {"Portal 2", "Team Fortress 2"},
    {"Portal 2", "Dota 2"},
]
counts = Counter()
for library in friends:
    counts.update(library)
print(counts.most_common(2))
```

```
[('Portal 2', 3), ('Dota 2', 2)]
```

# Два состава и цены на распродаже

```
l4d1 = {"Bill", "Zoey", "Louis", "Francis"}
l4d2 = {"Coach", "Ellis", "Nick", "Rochelle"}
print(len(l4d1 | l4d2), l4d1 & l4d2)

before = {"Portal": 9.99, "Portal 2": 9.99, "Left 4 Dead 2": 9.99}
after = {"Portal": 1.99, "Portal 2": 9.99, "Left 4 Dead 2": 1.99}
print(sorted(before.keys() & after.keys()))
print(sorted(before.items() - after.items()))
```

```
8 set()
['Left 4 Dead 2', 'Portal', 'Portal 2']
[('Left 4 Dead 2', 9.99), ('Portal', 9.99)]
```



# Снять лот прямо в цикле

```
lots = {1: "Arms of Desolation", 2: "Magus Apex", 3: "Kilowatt Case"}
try:
    for lot_id in lots:
        if lot_id == 2:
            del lots[lot_id]
except RuntimeError as error:
    print("RuntimeError:", error)
print(lots)
```

# Во время обхода ключи не удаляют и не добавляют

```
lots = {1: "Arms of Desolation", 2: "Magus Apex", 3: "Kilowatt Case"}
try:
    for lot_id in lots:
        if lot_id == 2:
            del lots[lot_id]
except RuntimeError as error:
    print("RuntimeError:", error)
print(lots)
```

```
RuntimeError: dictionary changed size during iteration
{1: 'Arms of Desolation', 3: 'Kilowatt Case'}
```

for lot_id in list(lots):

# В JSON ключи — строки

```
import json

answer = '{"730": {"success": true, "data": {"name": "Counter-Strike 2"}}}'
data = json.loads(answer)
print(list(data))
print("730" in data, 730 in data)
print(data["730"]["data"]["name"])
```

```
['730']
True False
Counter-Strike 2
```